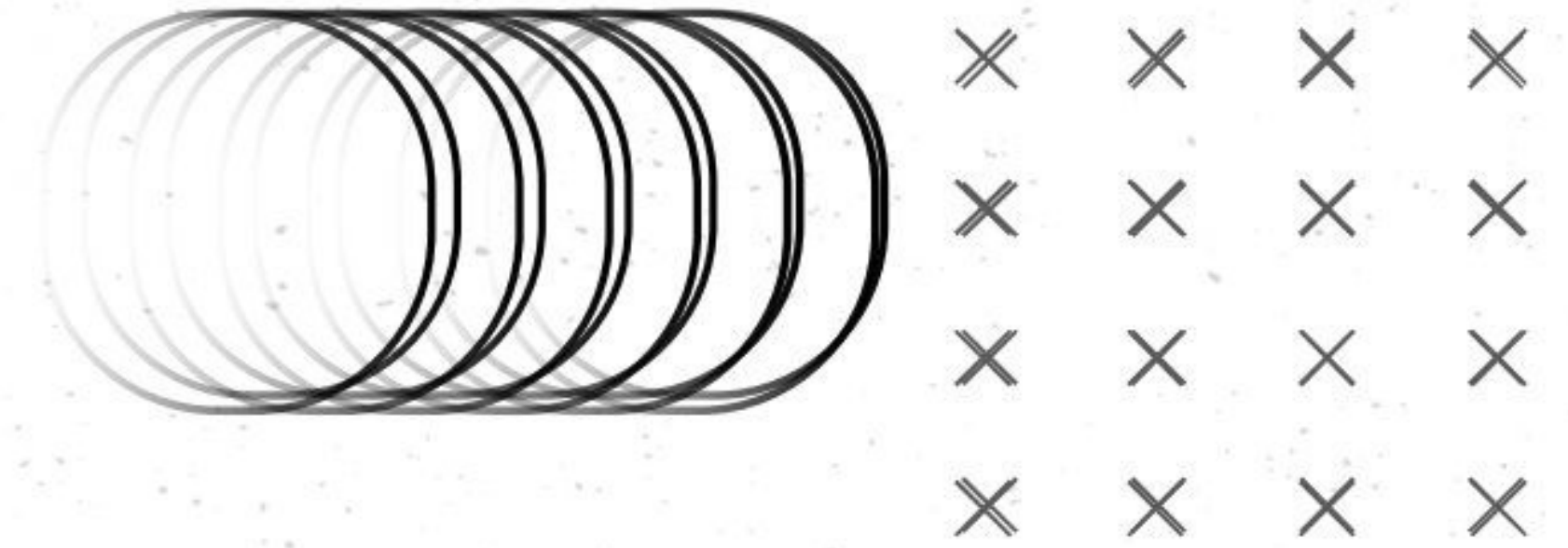




Data structure

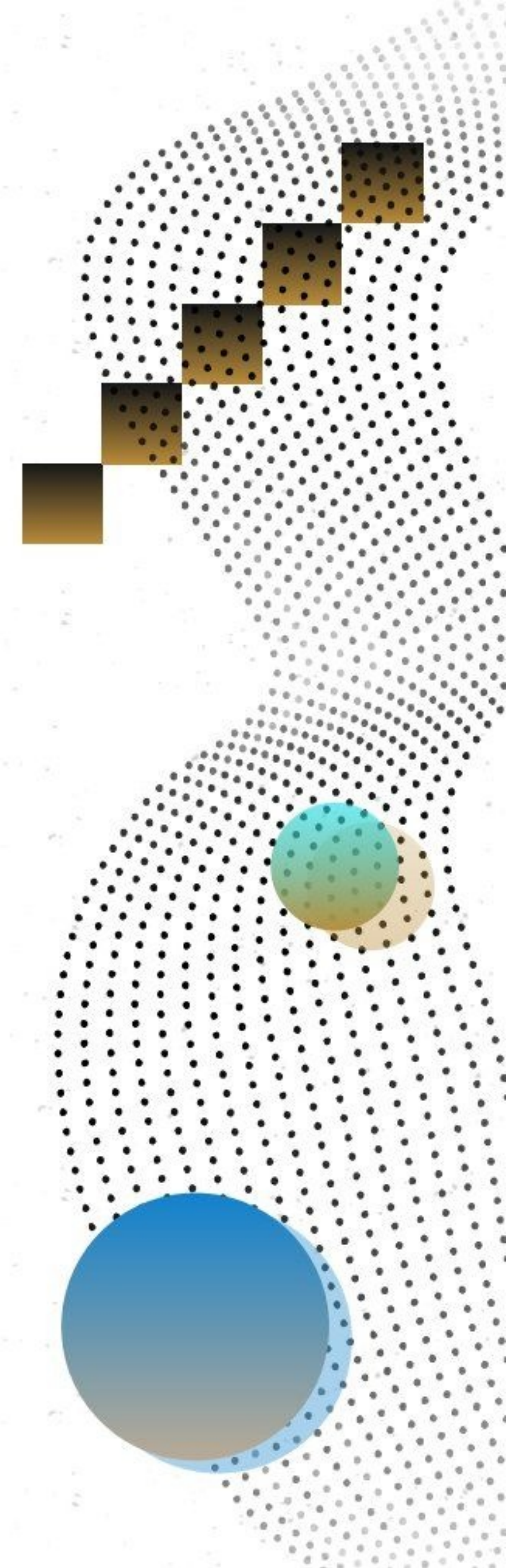
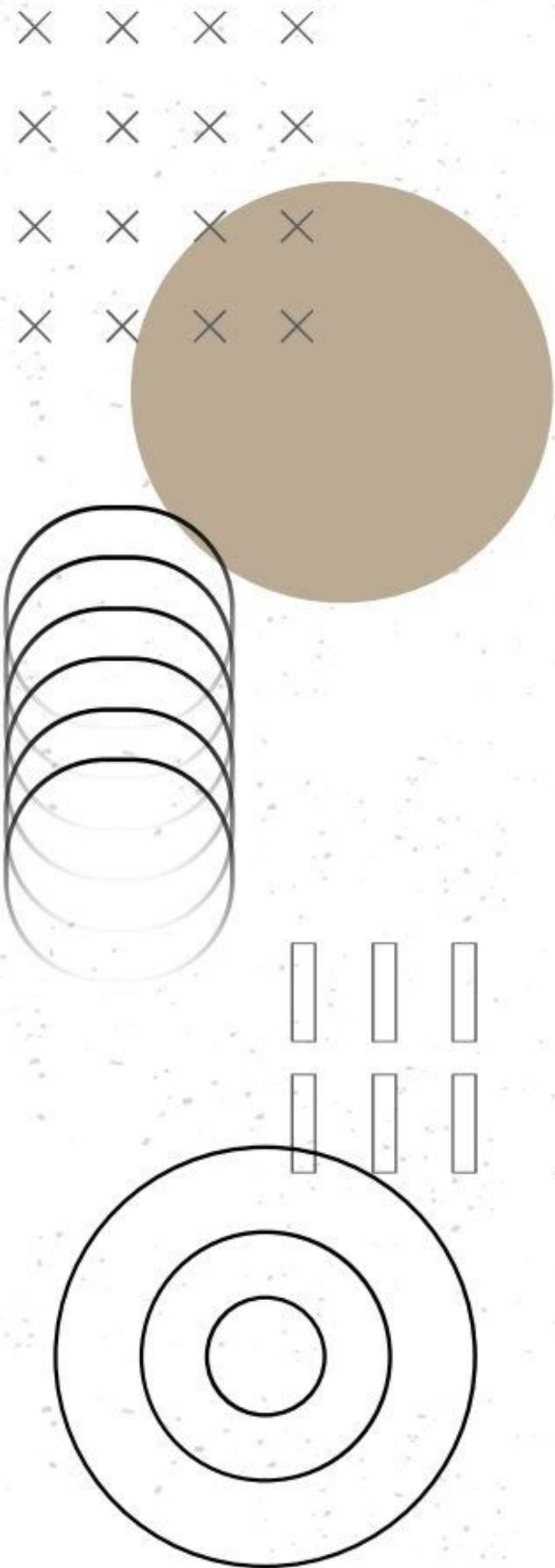
النخعي

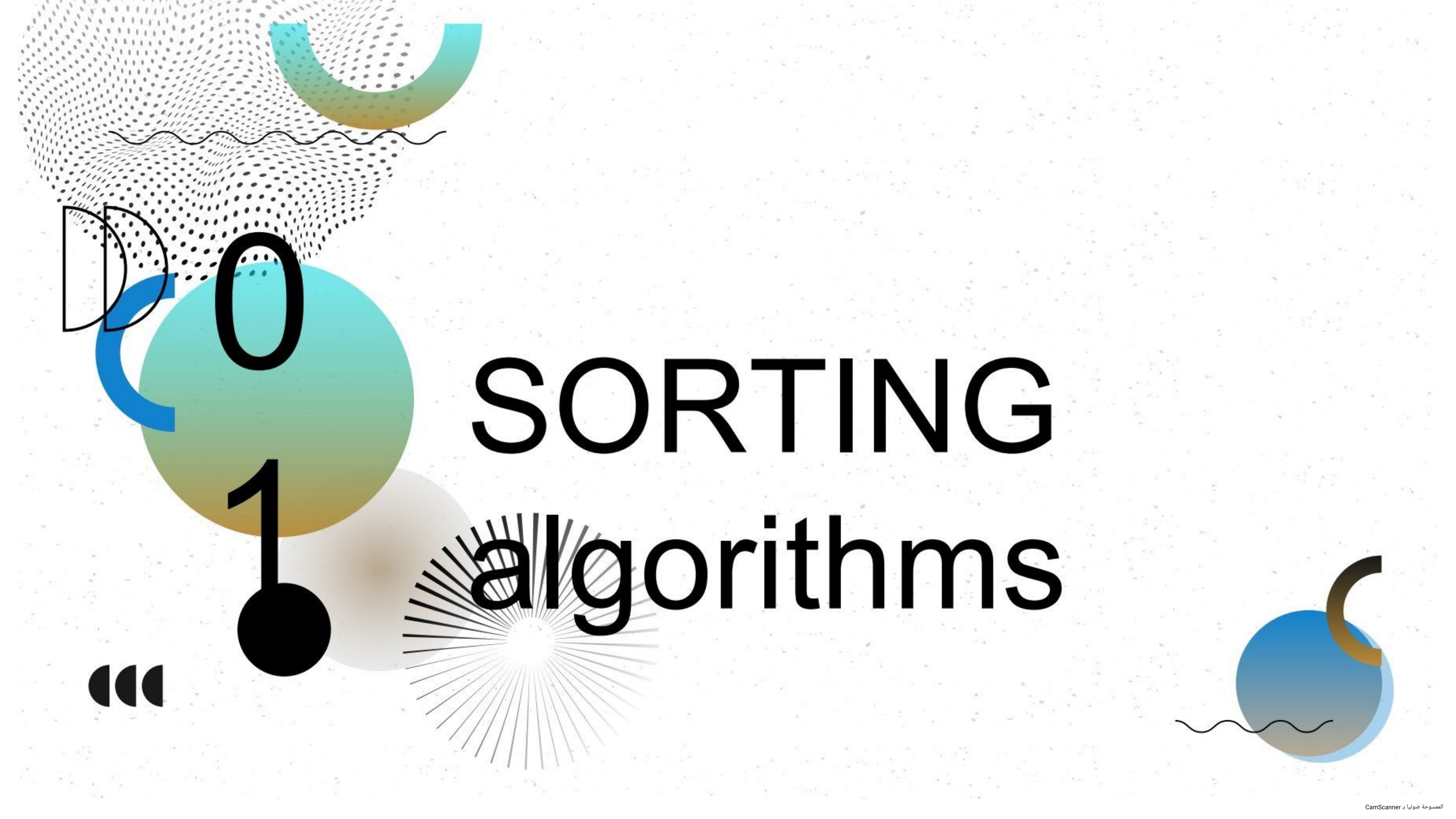
LECTURE " 5 " : SORT & SEARCH
ALGORITHMS



WHAT IS AN ALGORITHM

An algorithm is a step-by-step procedure or set of rules designed to solve a specific problem or perform a task. Algorithms are fundamental to

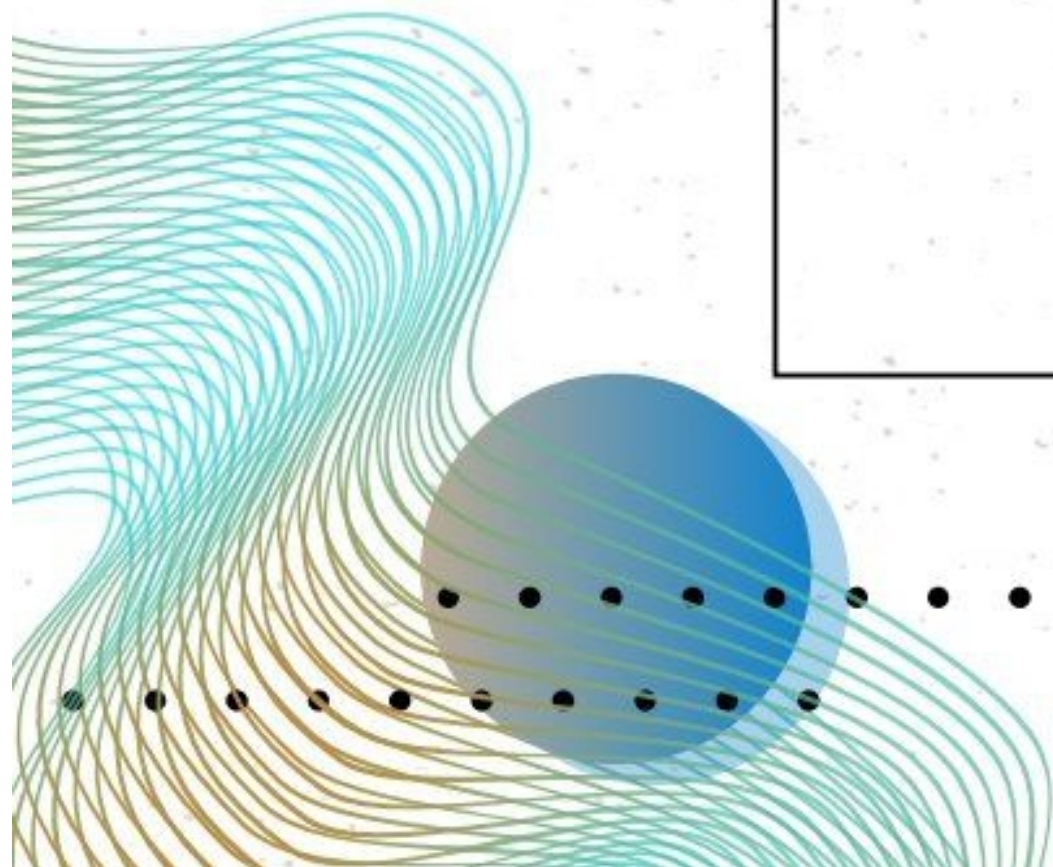




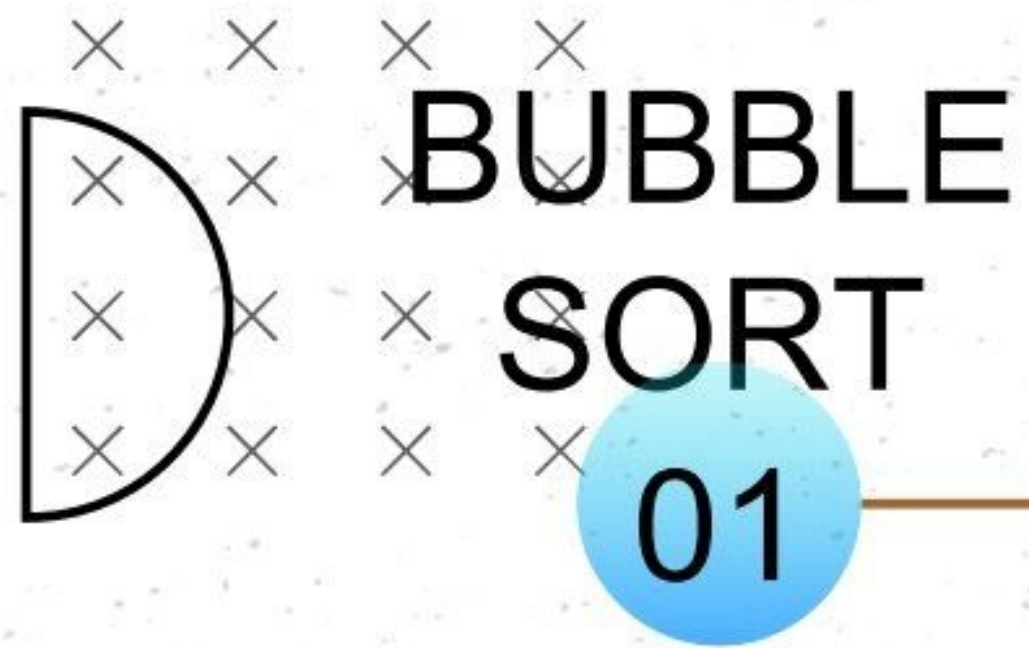
Sorting algorithms



A sorting algorithm is a method used to arrange elements in a list or array in a specific order, typically ascending or descending. Sorting helps organize data for easier searching, analysis, or display.



SORTING ALGORITHMS

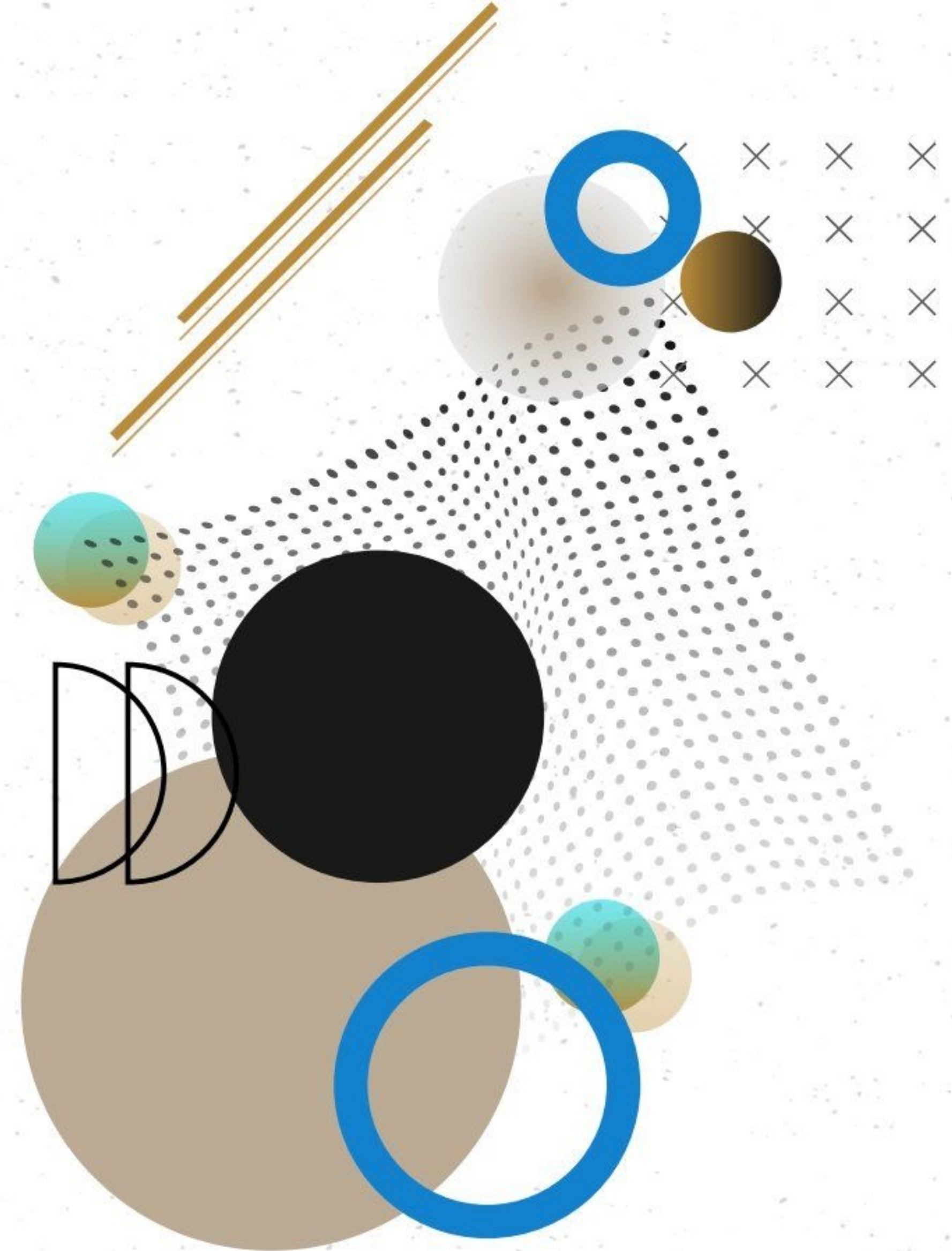


1. bubble

sort

Compare each element with
the next and swap if out of
order.

Repeat until the entire list is
sorted.



BUBBLE SORT

j $j+1$



$j=0$
 $i=0$



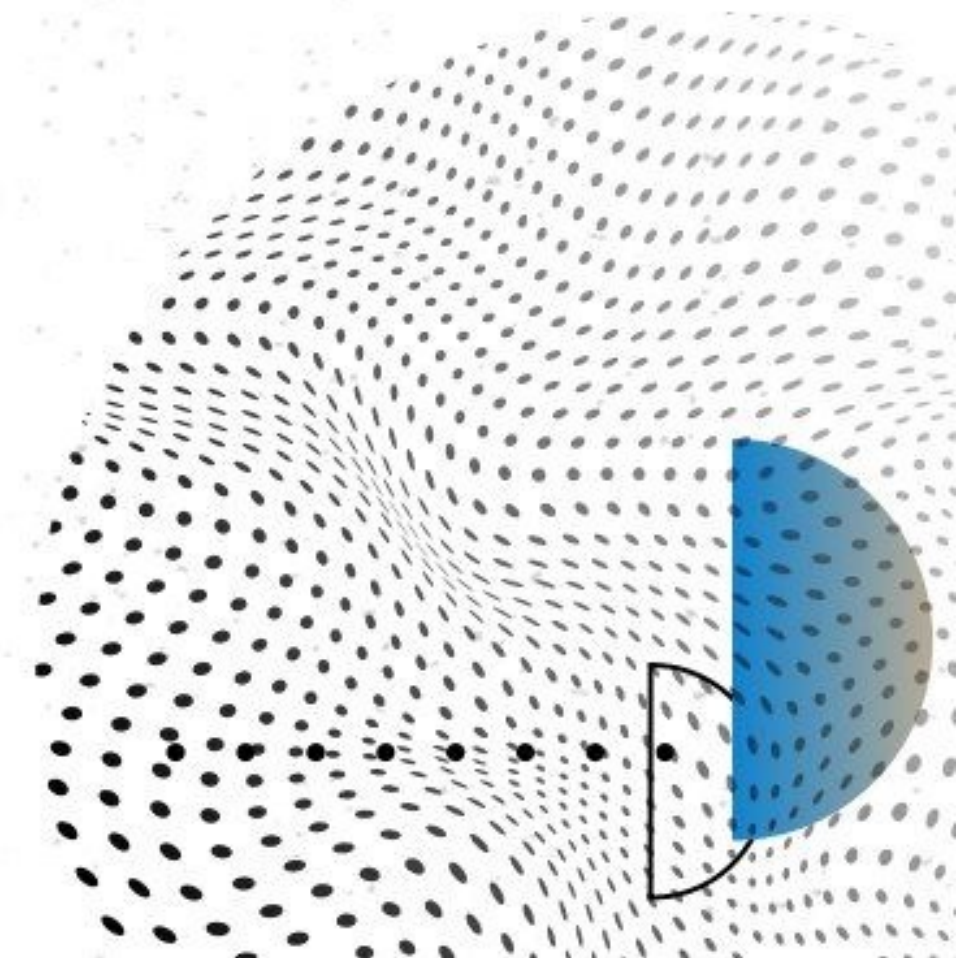
BUBBLE SORT Pseudocode

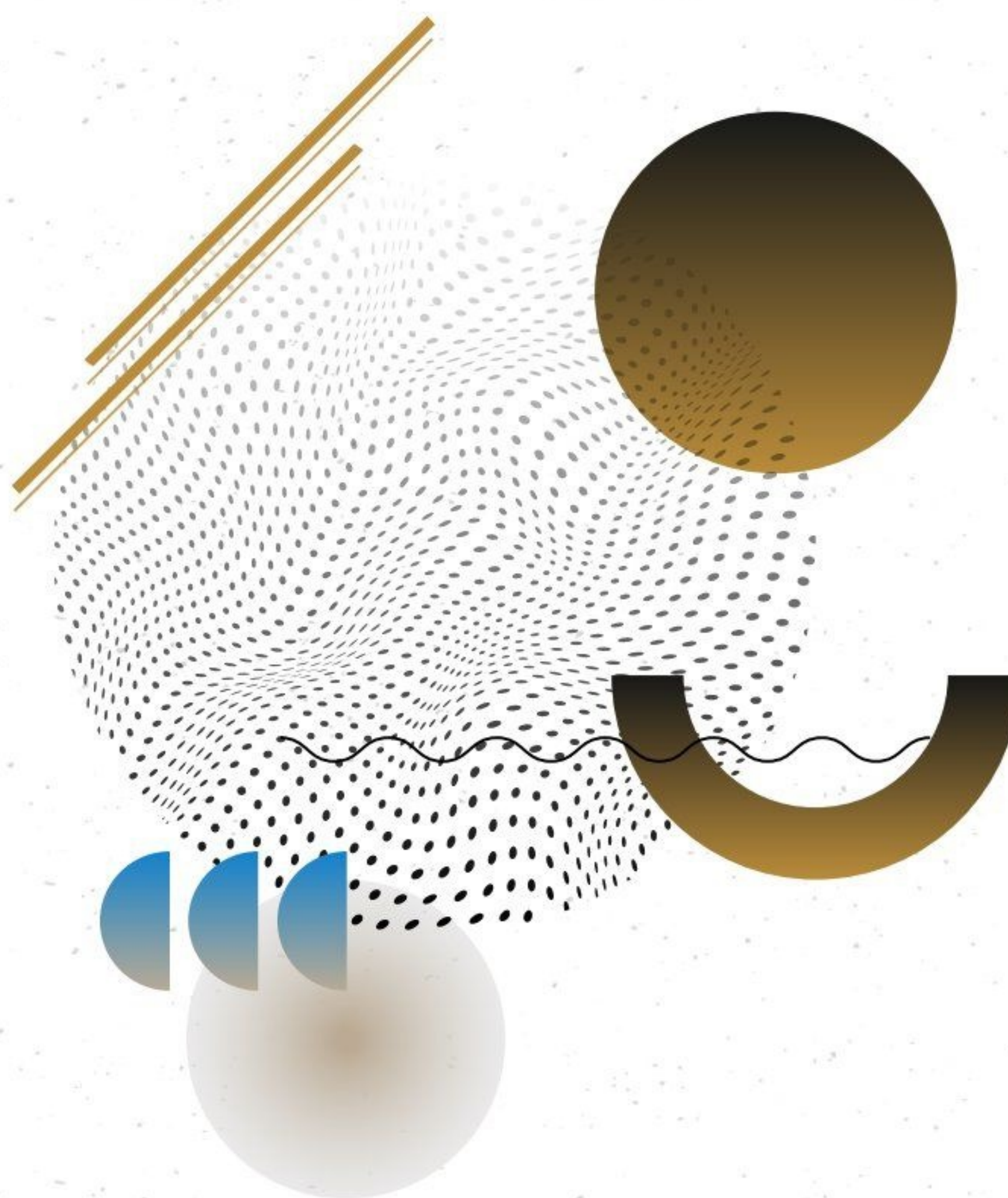
time complexity



```
BubbleSort(array):  
  n = length of array  
  for i from 0 to n-1:  
    for j from 0 to n-i-2:  
      if array[j] > array[j+1]:  
        swap array[j] and array[j+1]
```

Best Case: $O(n)$ (already sorted, with optimization)
Average/Worst Case: $O(n^2)$
Space Complexity: $O(1)$





2. selection sort

In each pass, find the
smallest element
and place it at the
start.



< T 2 >

SELECTION SORT

i_{small}
↓

4	7	3	1	8	6	2
---	---	---	---	---	---	---

↑
 j



SELECTION SORT Pseudocode

time complexity

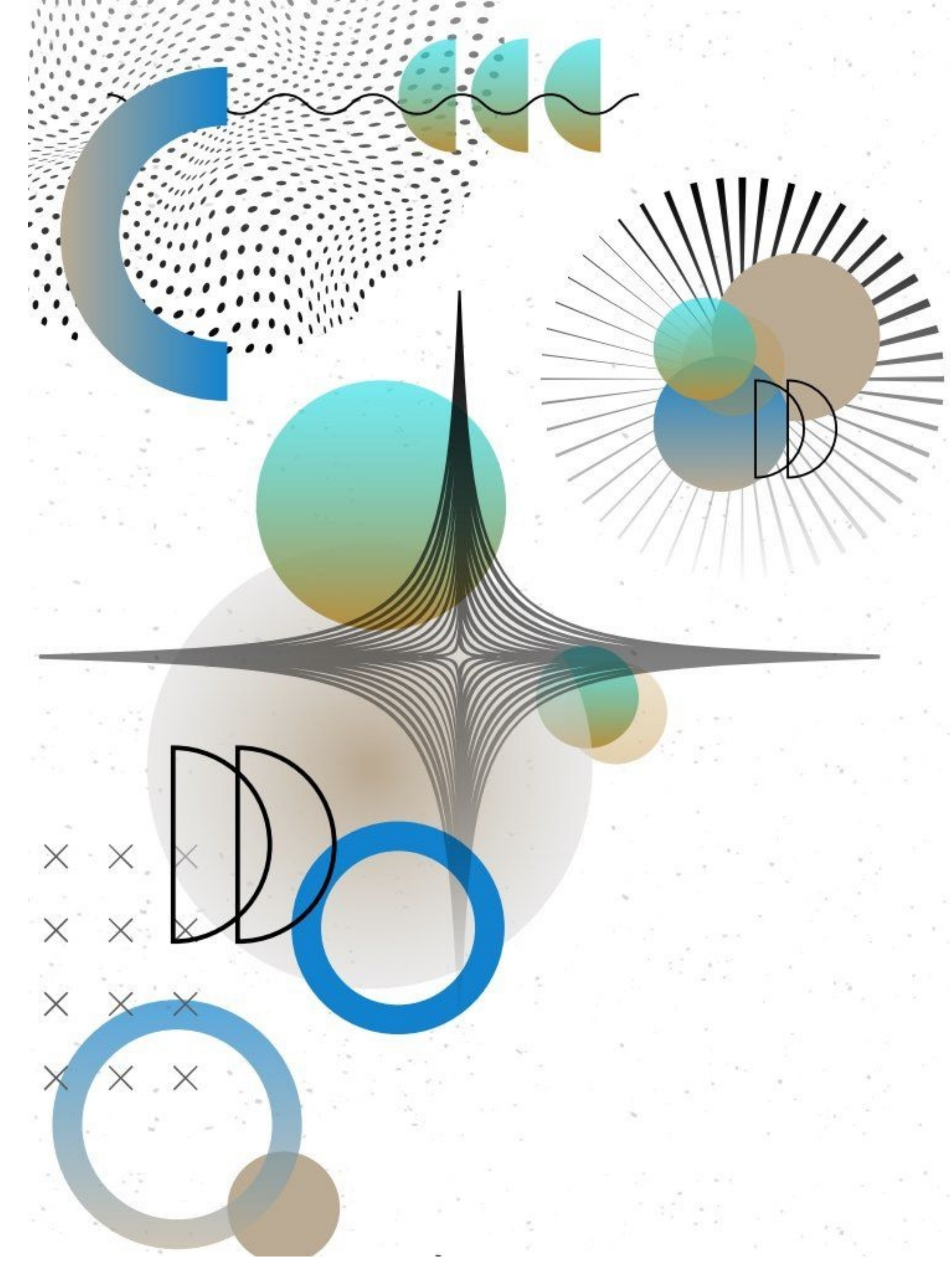


```
SelectionSort(array):  
  n = length of array  
  for i from 0 to n-1:  
    minIndex = i  
    for j from i+1 to n-1:  
      if array[j] < array[minIndex]:  
        minIndex = j  
    swap array[i] and array[minIndex]
```

Best/Worst/Average Case: $O(n^2)$

Space Complexity: $O(1)$

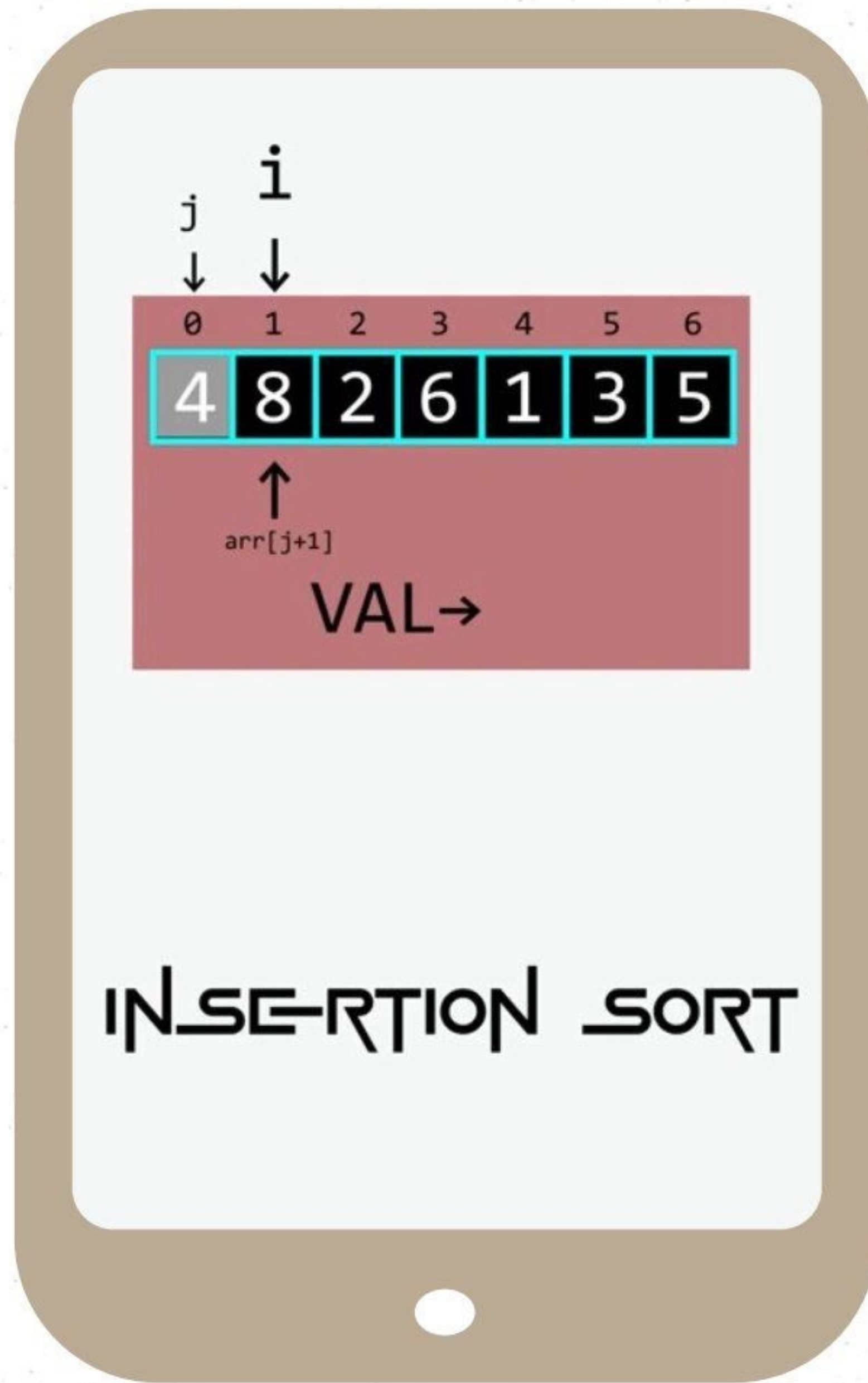
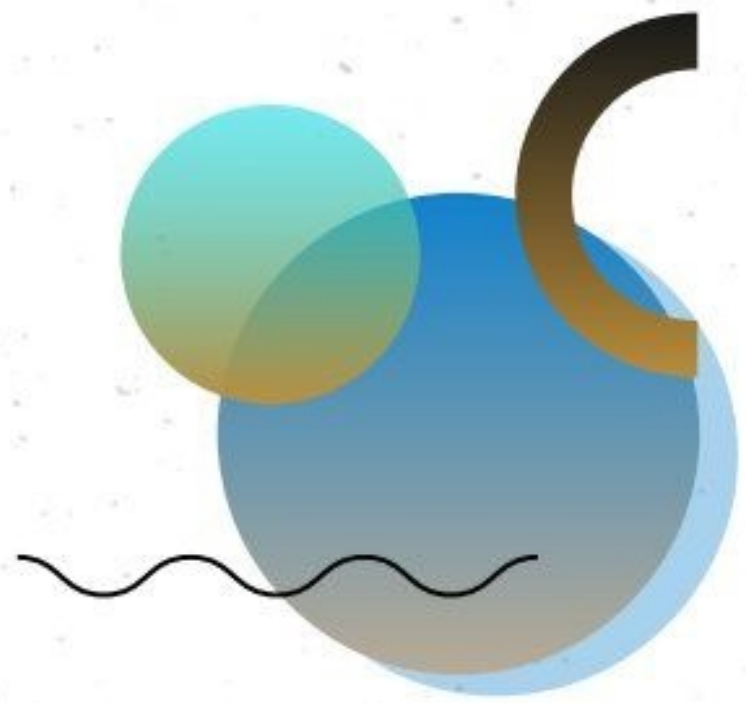




3. insertion sort

Insert each element
into its correct
position in the sorted
part.







INSERTION SORT Pseudo time complexity

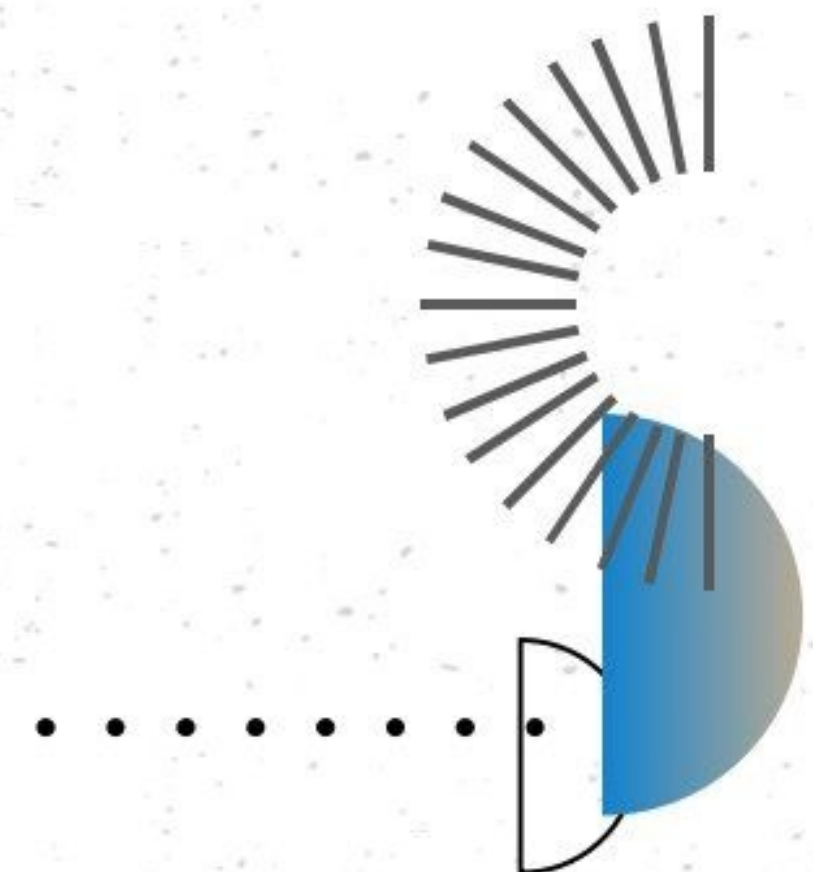


```
InsertionSort(array):  
  n = length of array  
  for i from 1 to n-1:  
    key = array[i]  
    j = i - 1  
    while j >= 0 and array[j] > key:  
      array[j+1] = array[j]  
      j = j - 1  
    array[j+1] = key
```

Best Case: $O(n)$ (already sorted)

Average/Worst Case: $O(n^2)$

Space Complexity: $O(1)$



4. Merge sort

- Split the list, sort each half, then merge them in order.

Merge Sort

7	5	1	3	4	6	2
---	---	---	---	---	---	---



MERGE SORT Pseudocode

time complexity

MergeSort(array):

if length of array > 1:

mid = length of array // 2

left = array[0:mid]

right = array[mid:]

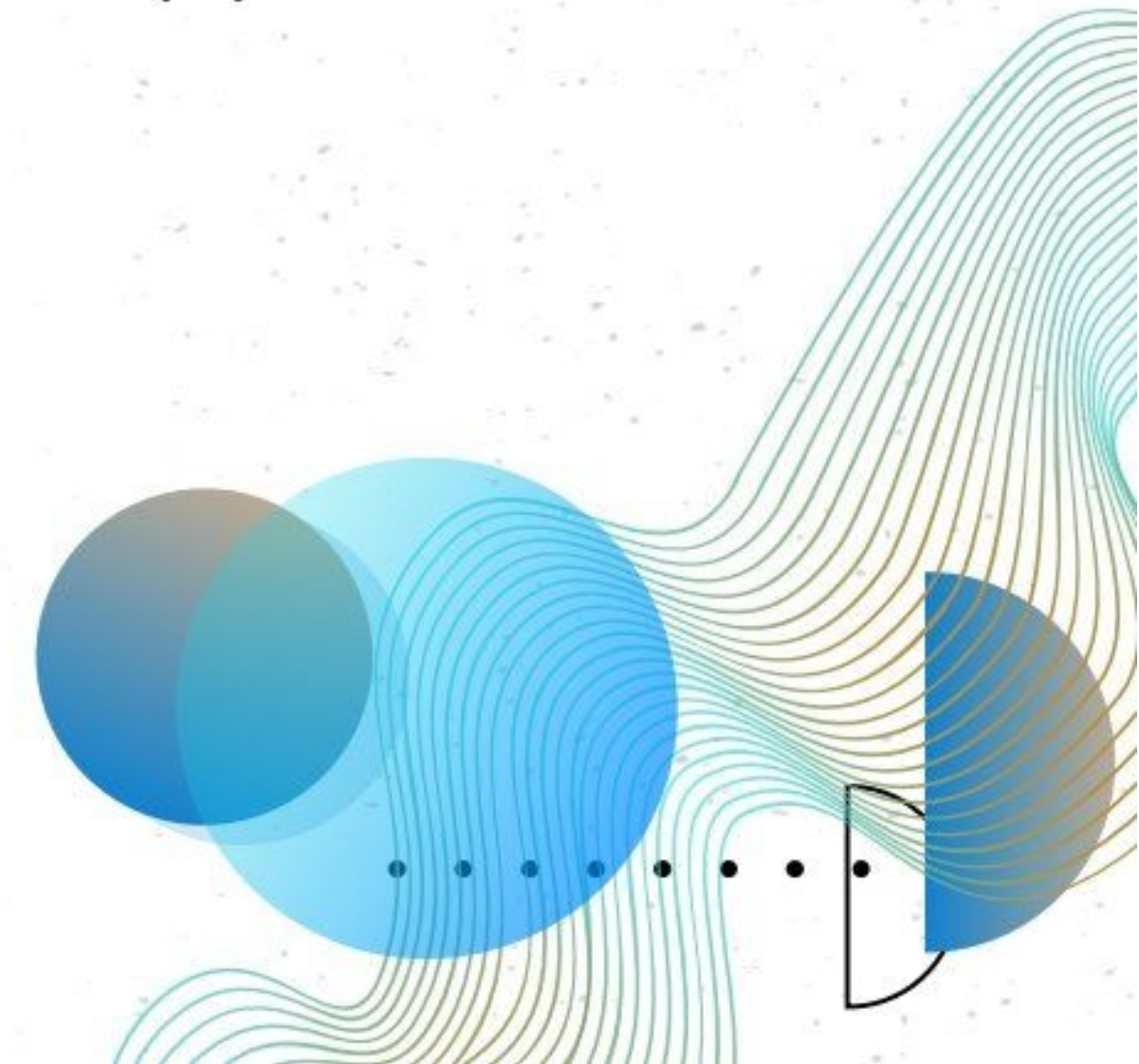
MergeSort(left)

MergeSort(right)

merge left and right into array

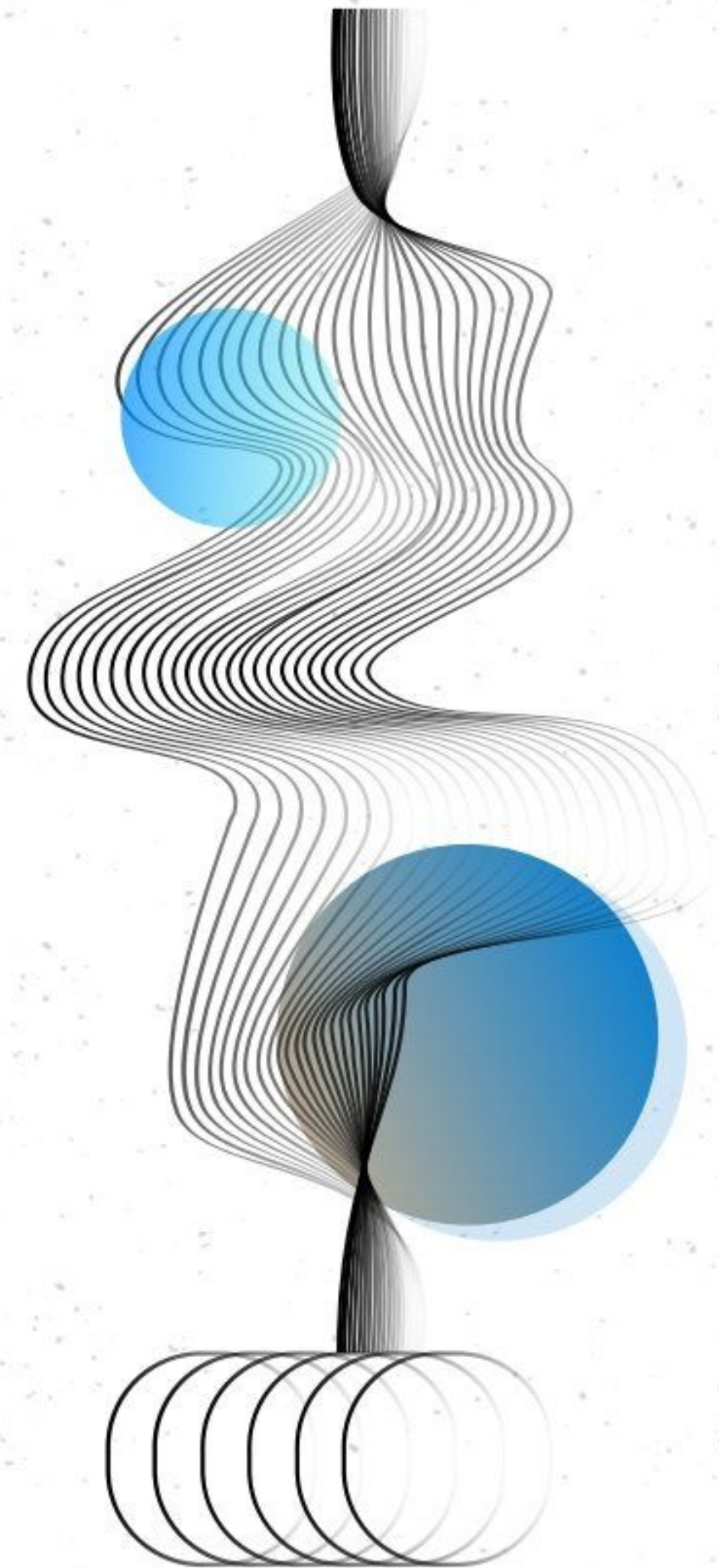
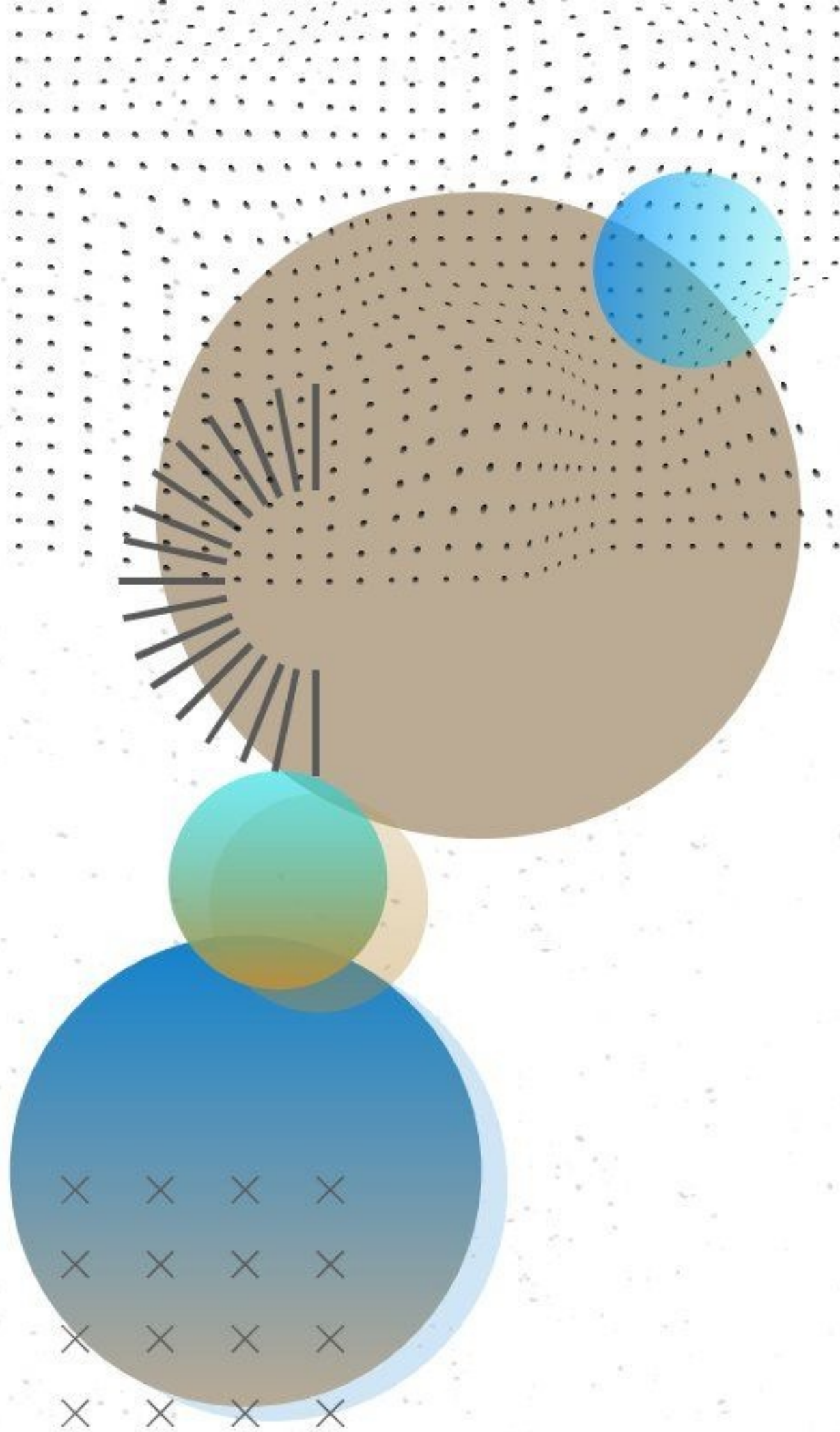
Best/Average/Worst Case: $O(n \log n)$

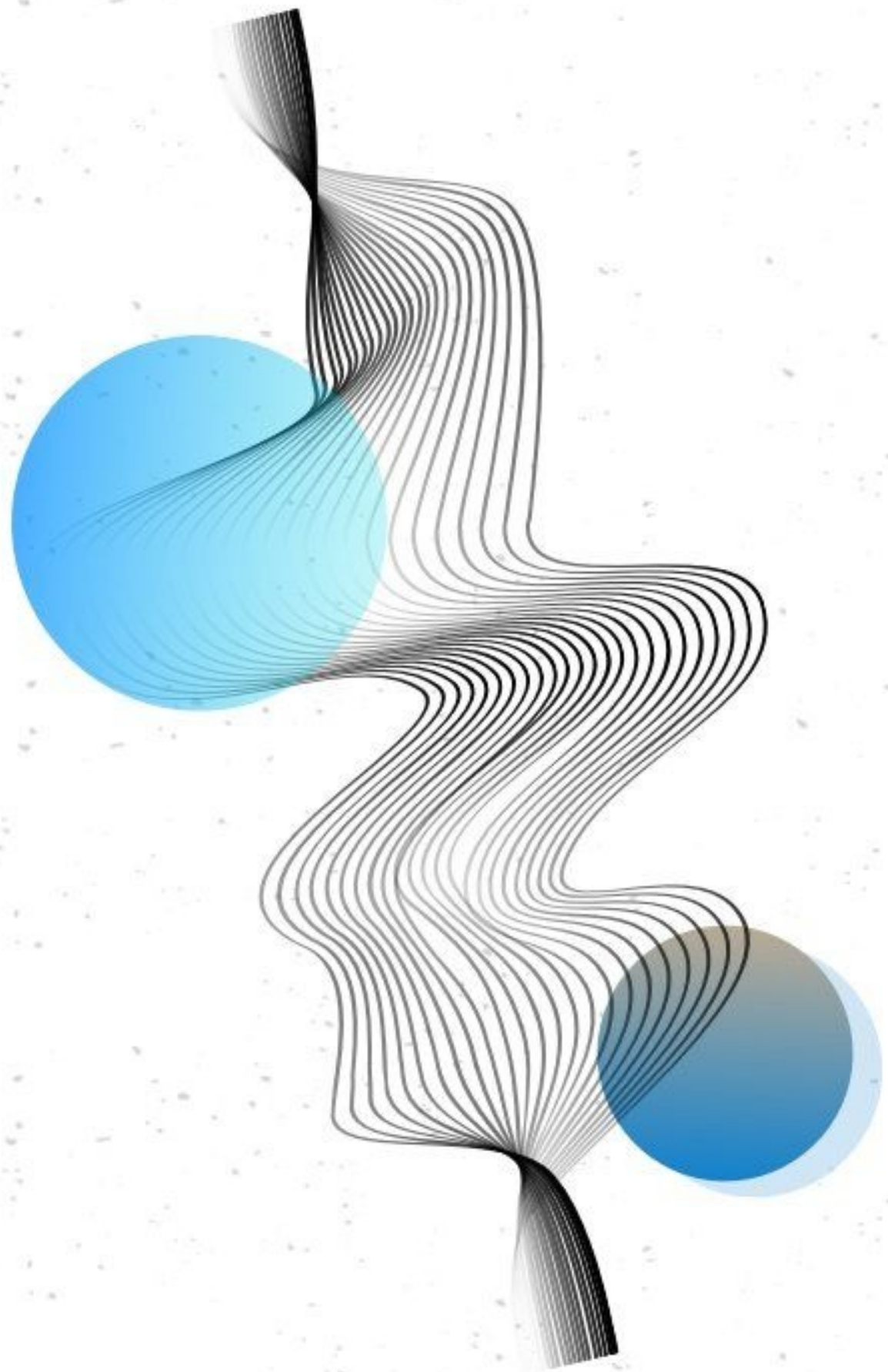
Space Complexity: $O(n)$



5. Quick sort

Choose a pivot, divide
elements into $<$ pivot
and $>$ pivot, then sort
recursively.





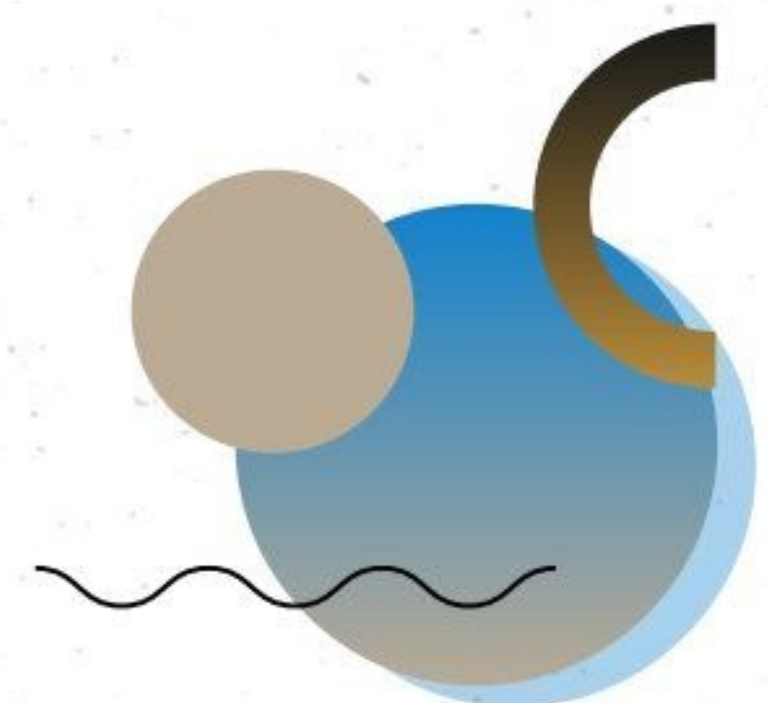
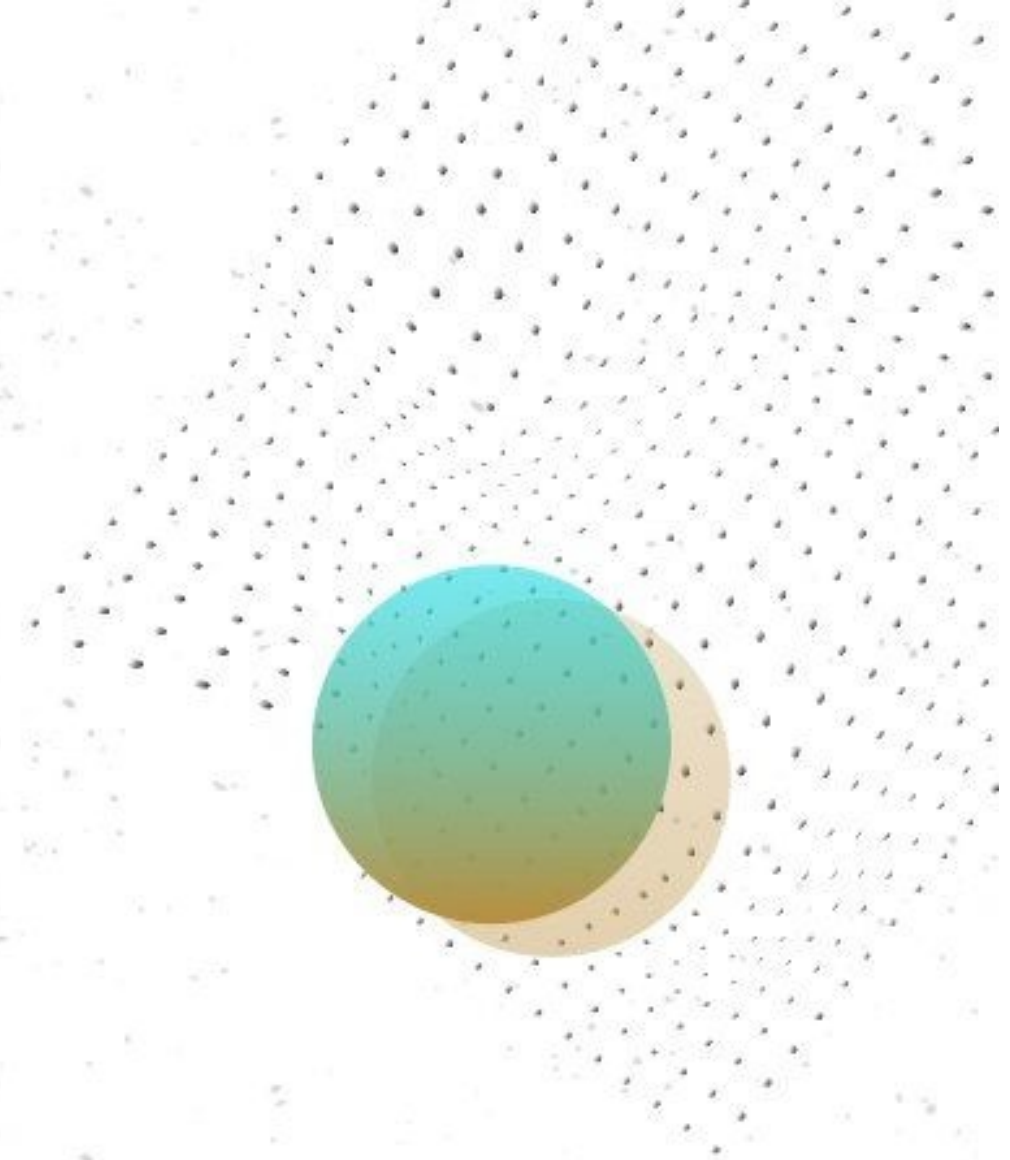
Quick Sort

i
↓

k
↓

pivot
↓

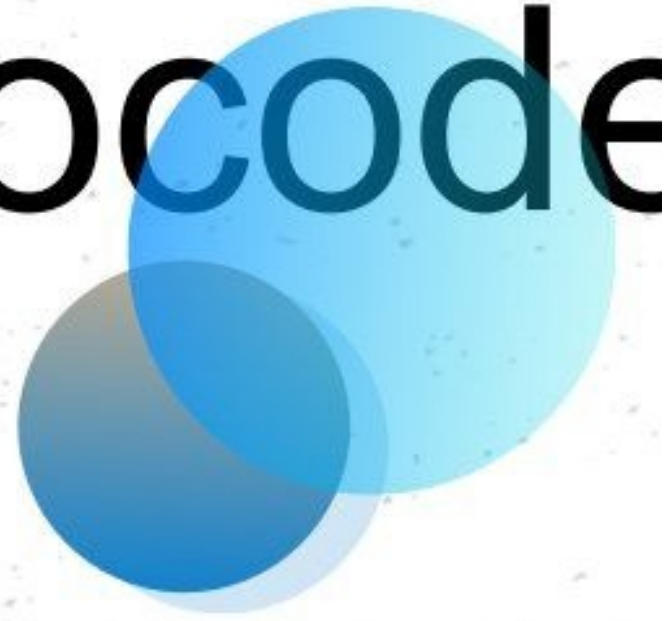
4	2	7	5	1	6	3
---	---	---	---	---	---	---





QUICK SORT Pseudocode

time complexity



QuickSort(array, low, high):

if low < high:

 pivotIndex = Partition(array, low, high)

 QuickSort(array, low, pivotIndex - 1)

 QuickSort(array, pivotIndex + 1, high)

Partition(array, low, high):

 pivot = array[high]

 i = low - 1

 for j from low to high - 1:

 if array[j] <= pivot:

 i = i + 1

 swap array[i] and array[j]

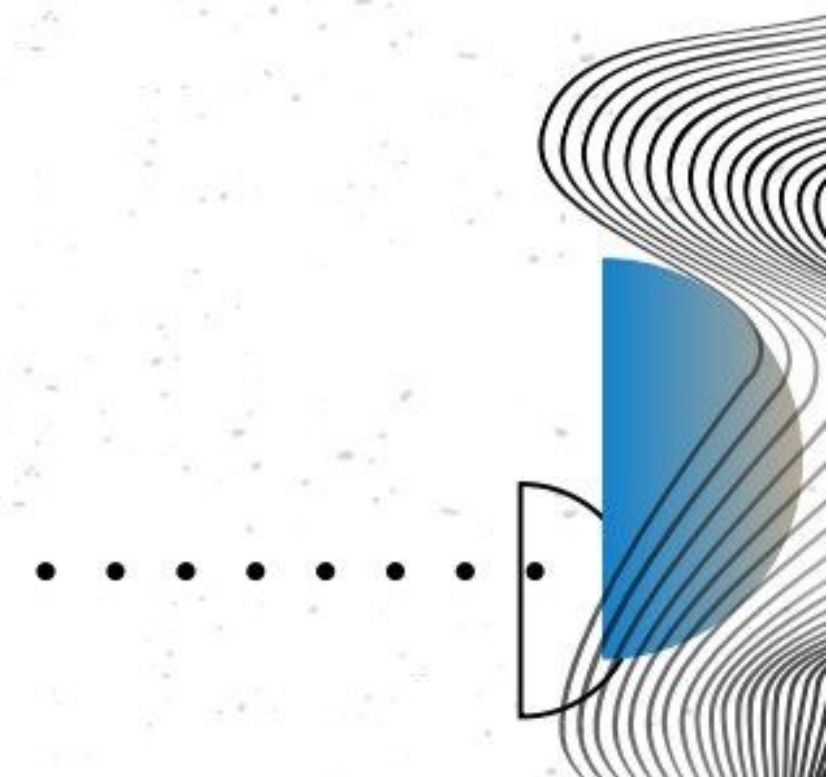
 swap array[i+1] and array[high]

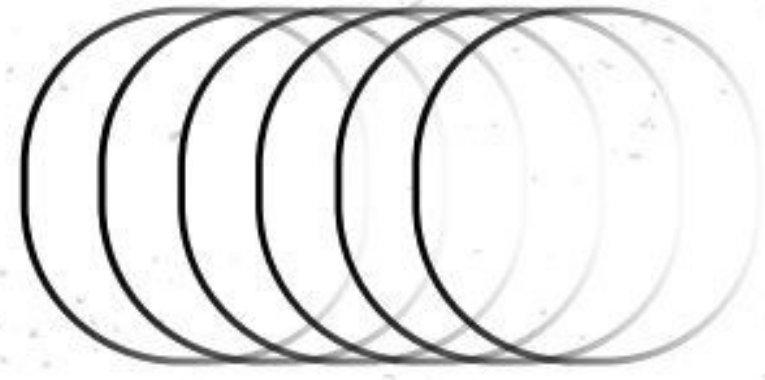
 return i + 1

Best/Average Case: $O(n \log n)$

Worst Case: $O(n^2)$ (bad pivot choice)

Space Complexity: $O(\log n)$ (recursive stack)





02

Searching

A structured method to locate a specific item in data structures like arrays, trees, or graphs.



< 0 1 >

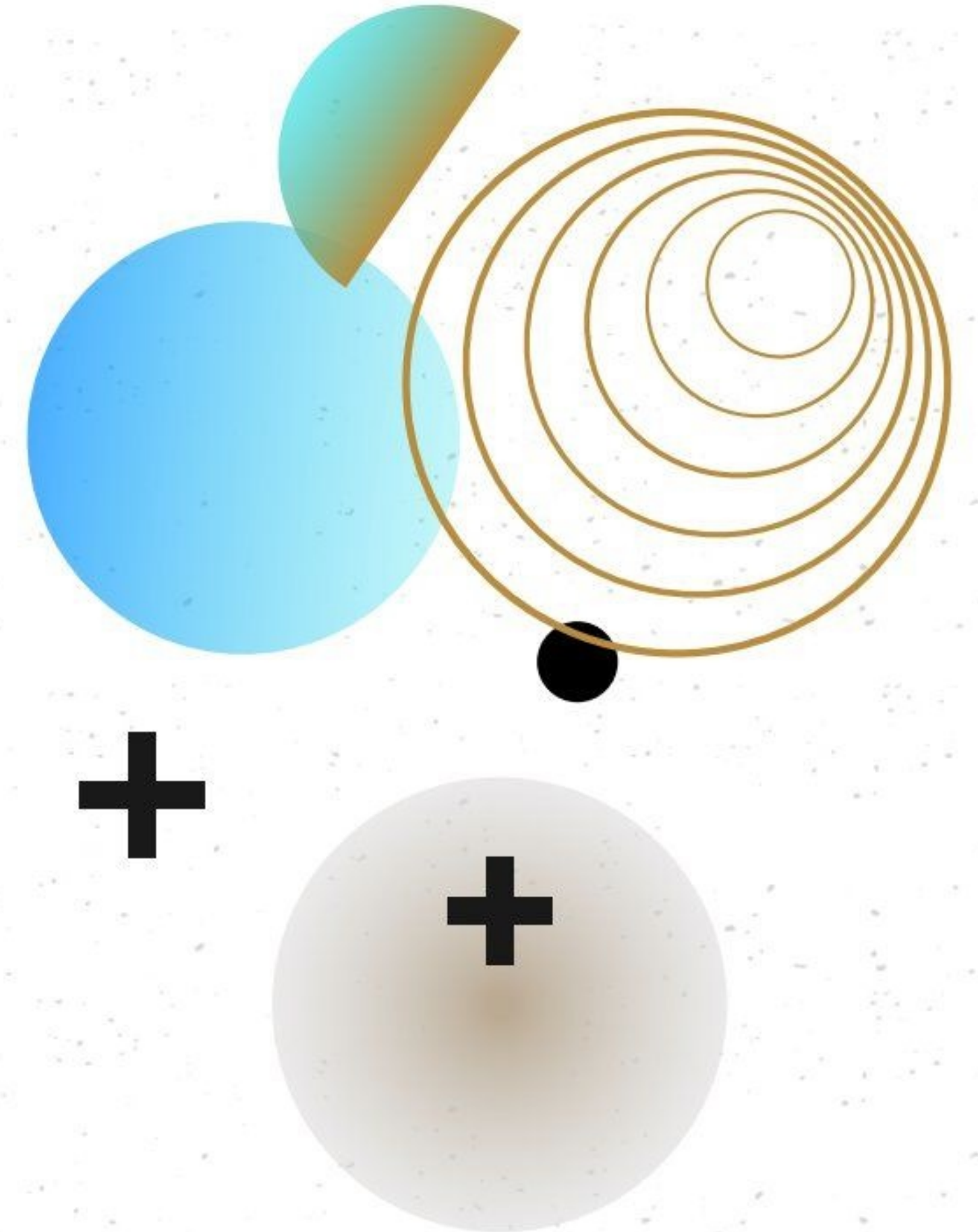




Table of contents

LINEAR
01 SEARCH

BINARY
SEARCH 02

03 BF SEARCH DF SEARCH 04

+

+

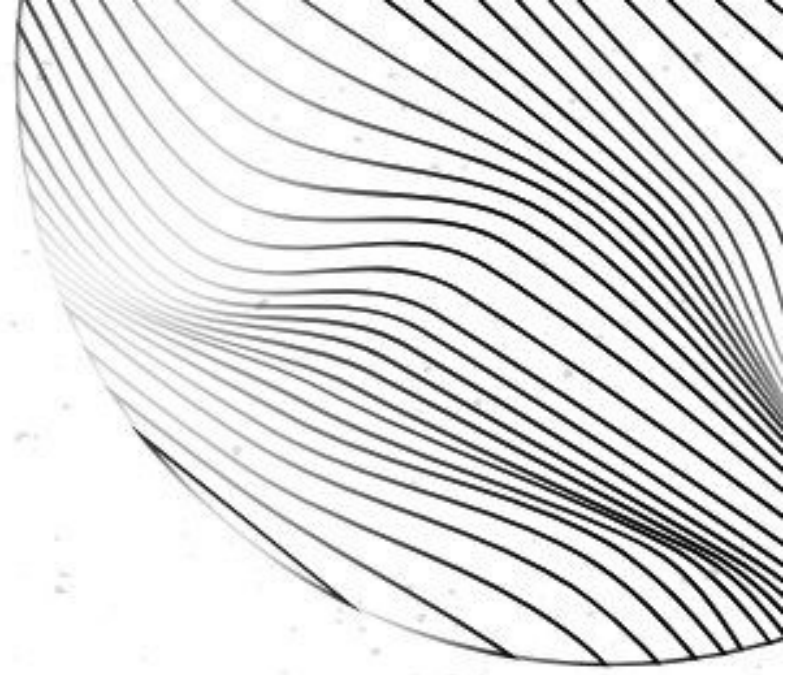
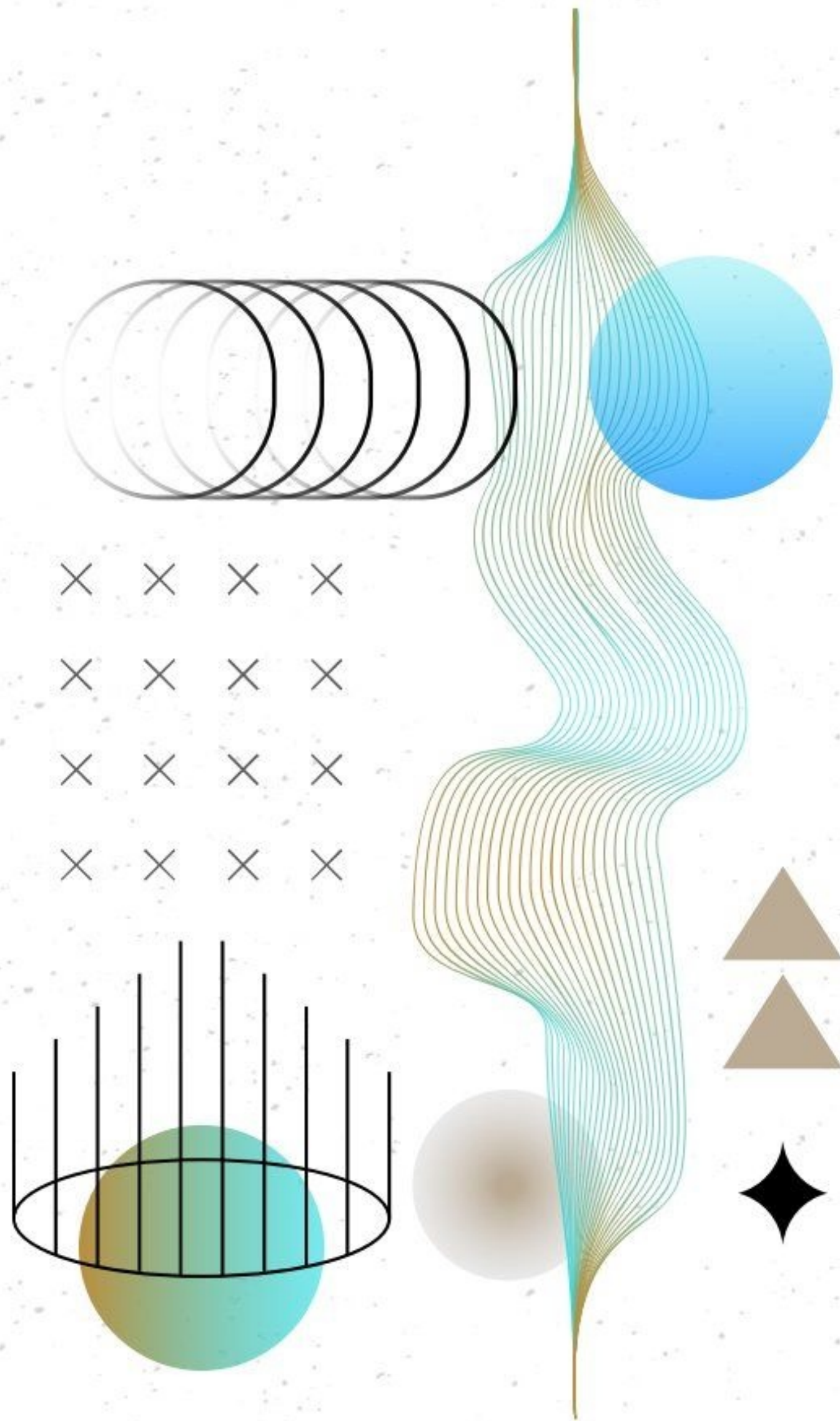
< T 0 >

⋮

1. LINEAR SEARCH

The thread class in .NET

C# is used to create and control a thread set its priority , and gets its status



LINEAR SEARCH

0 1 2 3 4 5 6 7 8

4 8 2 6 5 7 3 9 0



LINEAR SEARCH Pseudocode

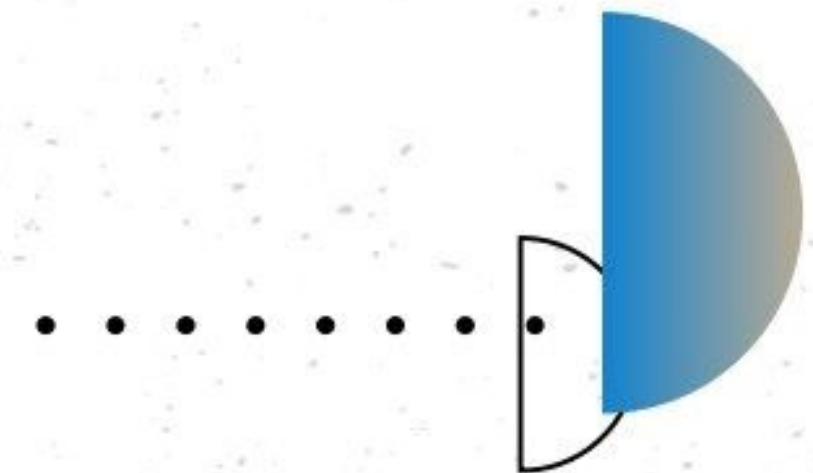
time complexity

```
LinearSearch(array, target):  
  for i from 0 to length of array - 1:  
    if array[i] == target:  
      return i // Target found at index i  
  return -1 // Target not found
```

Best Case: $O(1)$ — The target is at the first position.

Worst Case: $O(n)$ — The target is at the last position or not present at all.

Average Case: $O(n)$ — On average, half the elements are checked before finding the target.



2. binary SEARCH

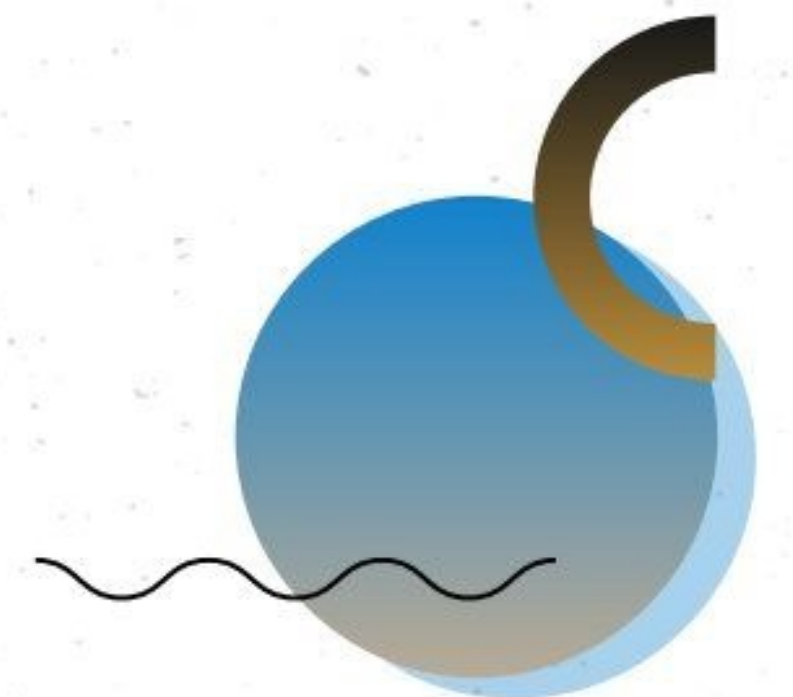
Efficient method for
finding an item in a
sorted array

< T 2 >

BINARY SEARCH

0 1 2 3 4 5 6 7

1 2 3 4 5 6 7 8





BINARY SEARCH Pseudo time complexity

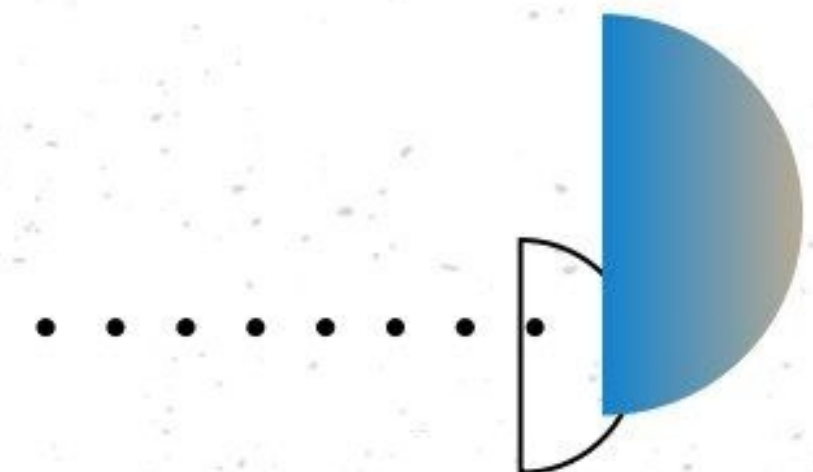


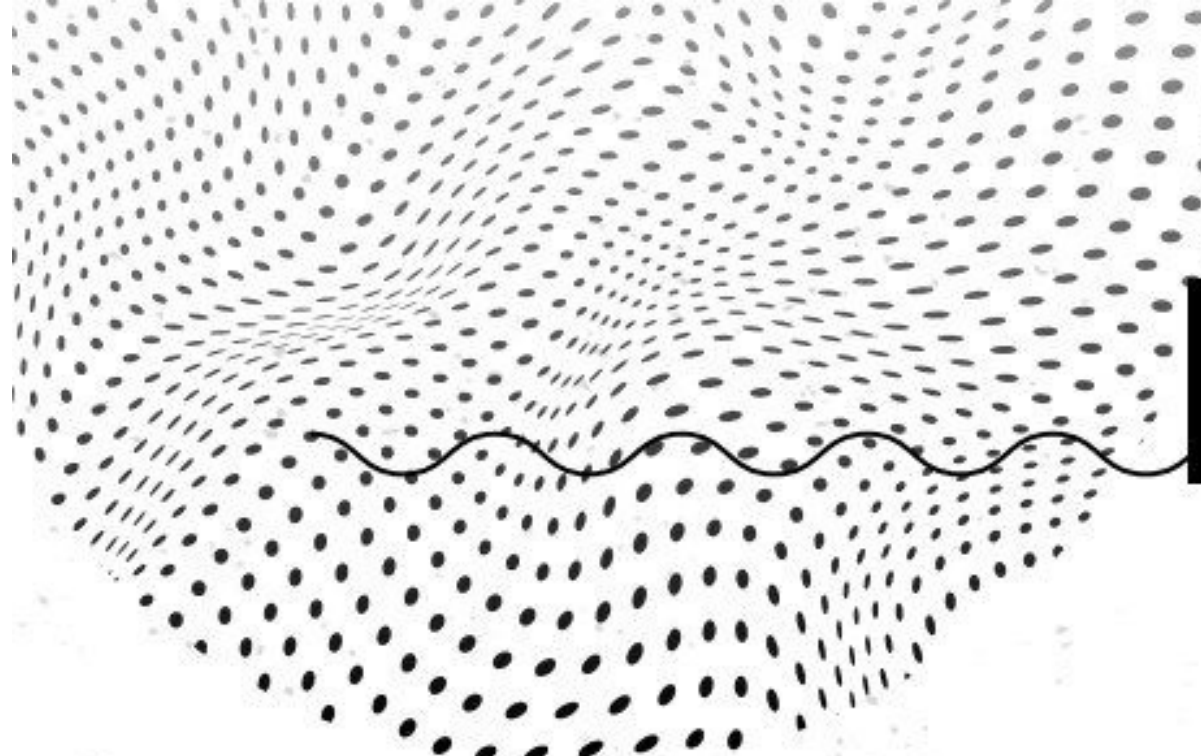
```
BinarySearch(array, target):  
  low = 0  
  high = length of array - 1  
  while low ≤ high:  
    mid = (low + high) // 2  
    if array[mid] == target:  
      return mid      // Target found  
    else if array[mid] < target:  
      low = mid + 1    // Search right  
    half  
    else:  
      high = mid - 1  // Search left half  
  
  return -1           // Target not found
```

Best Case: $O(1)$ — The target is at the middle on the first check.

Worst Case: $O(\log n)$ — The search space is halved each time.

Average Case: $O(\log n)$ — Logarithmic time due to divide-and-conquer.





Introduction to Graphs

Graph = collection of vertices (nodes) and edges (connections)

Types:

Undirected Graph → edges have no direction

Directed Graph → edges have a direction

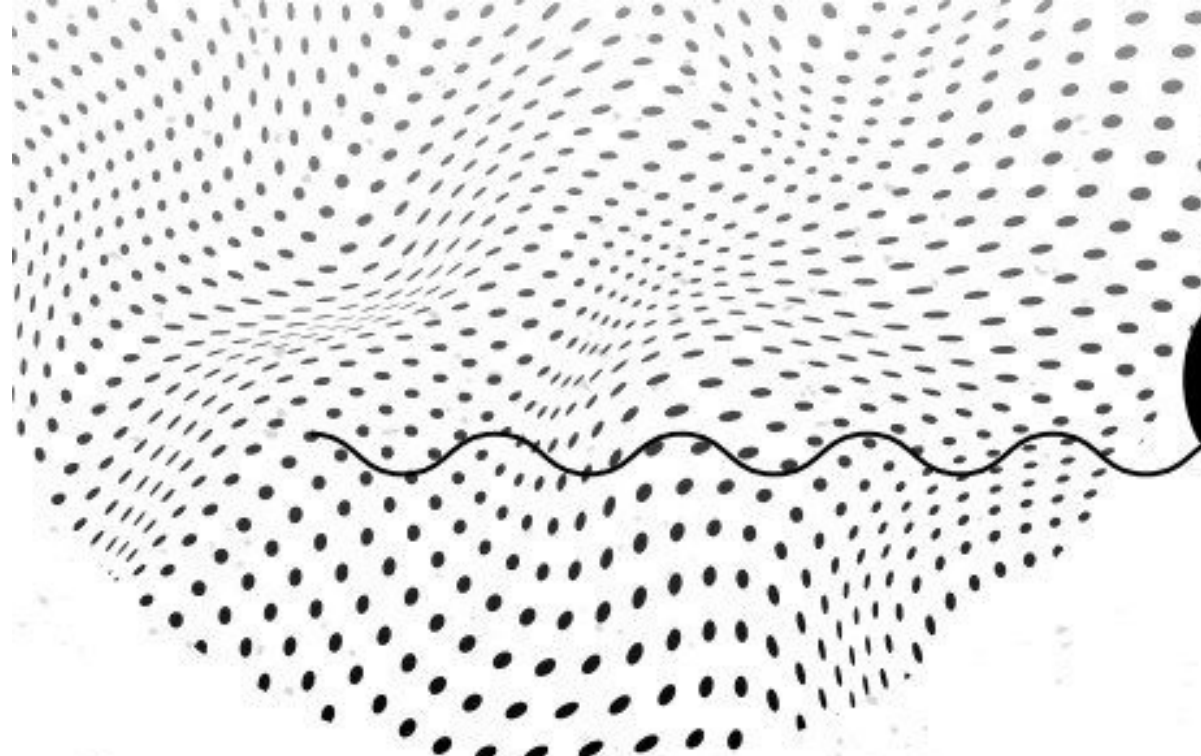
Examples:

Social networks

Road maps

Web pages links





Graph Terminology

Vertex / Node = element

Edge = connection

Degree = number of edges connected to a node

Adjacent = connected nodes

Path = sequence of nodes connected by edges

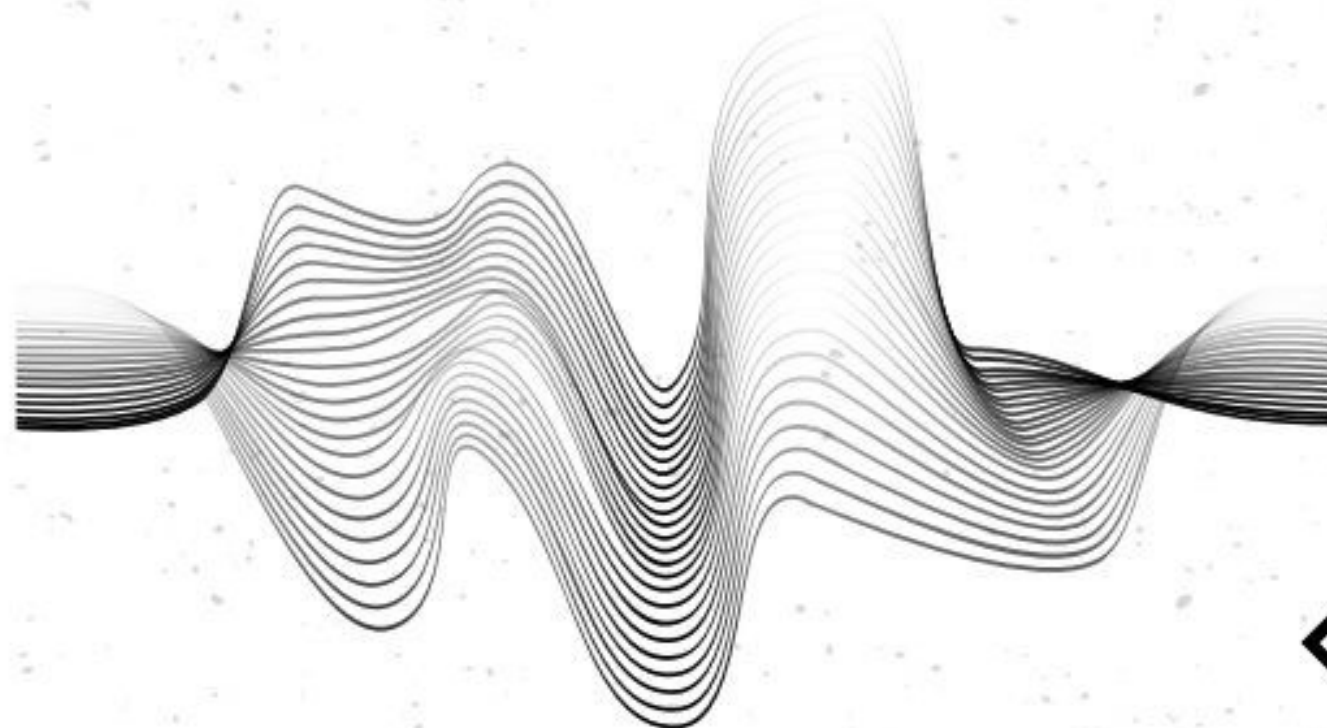
Cycle = path that starts and ends at the same node



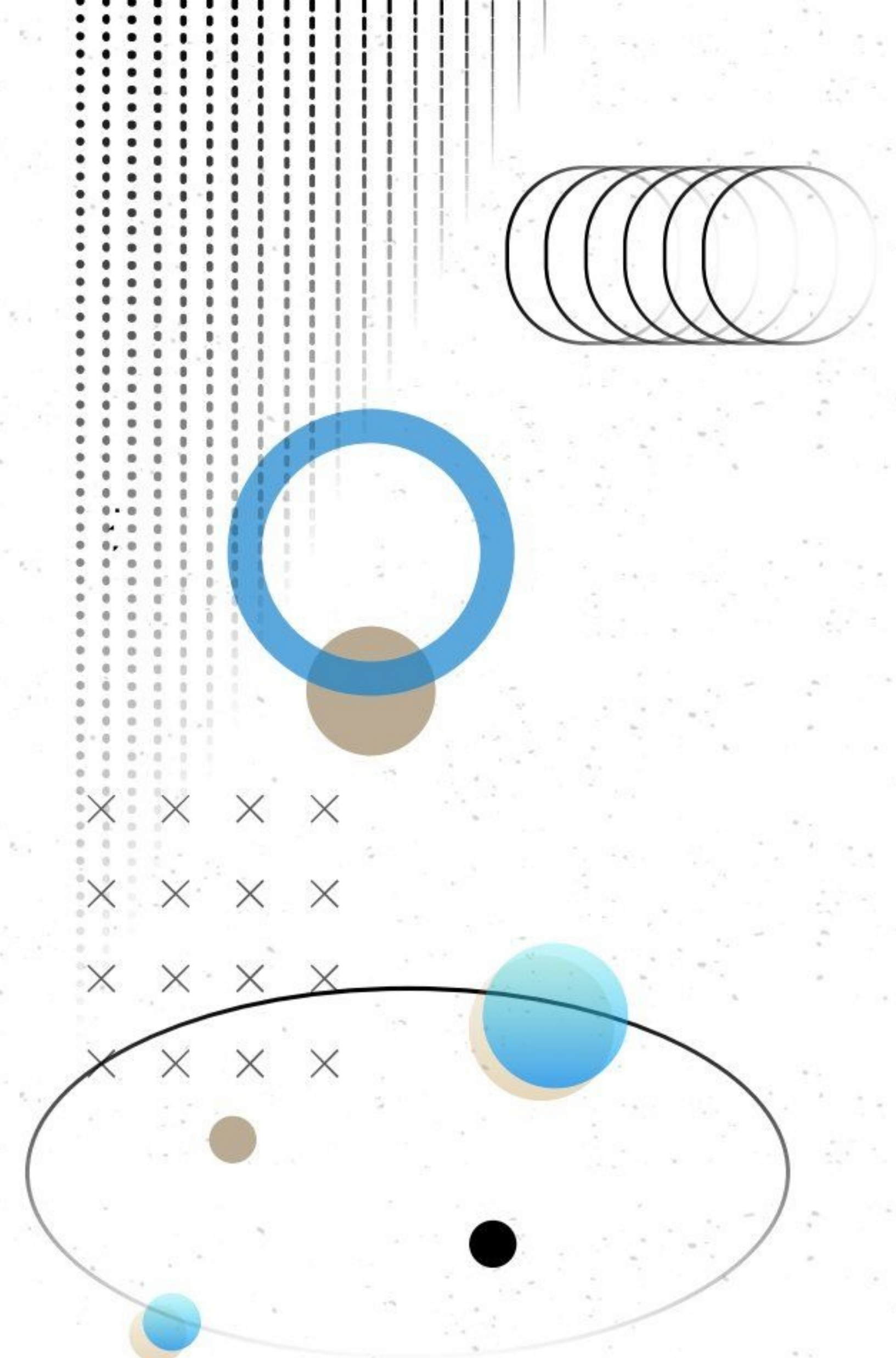
3. BF

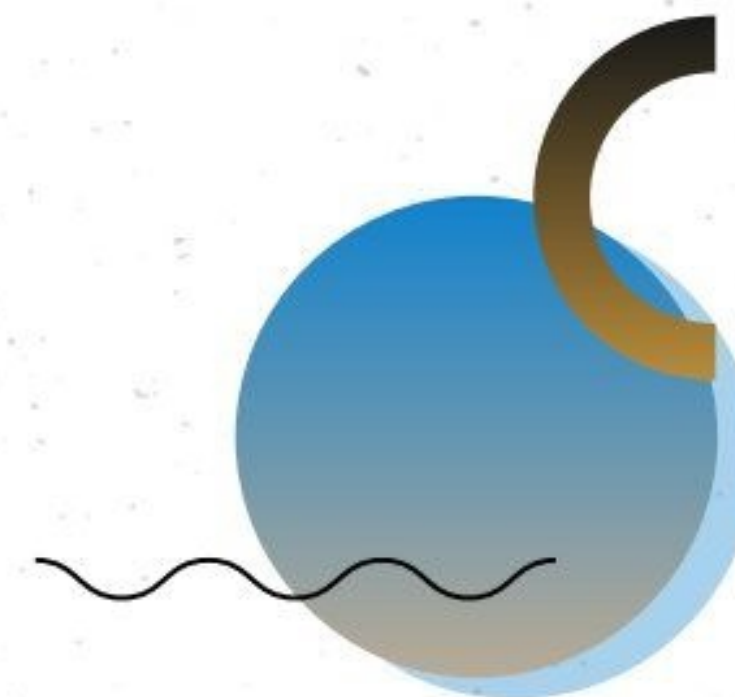
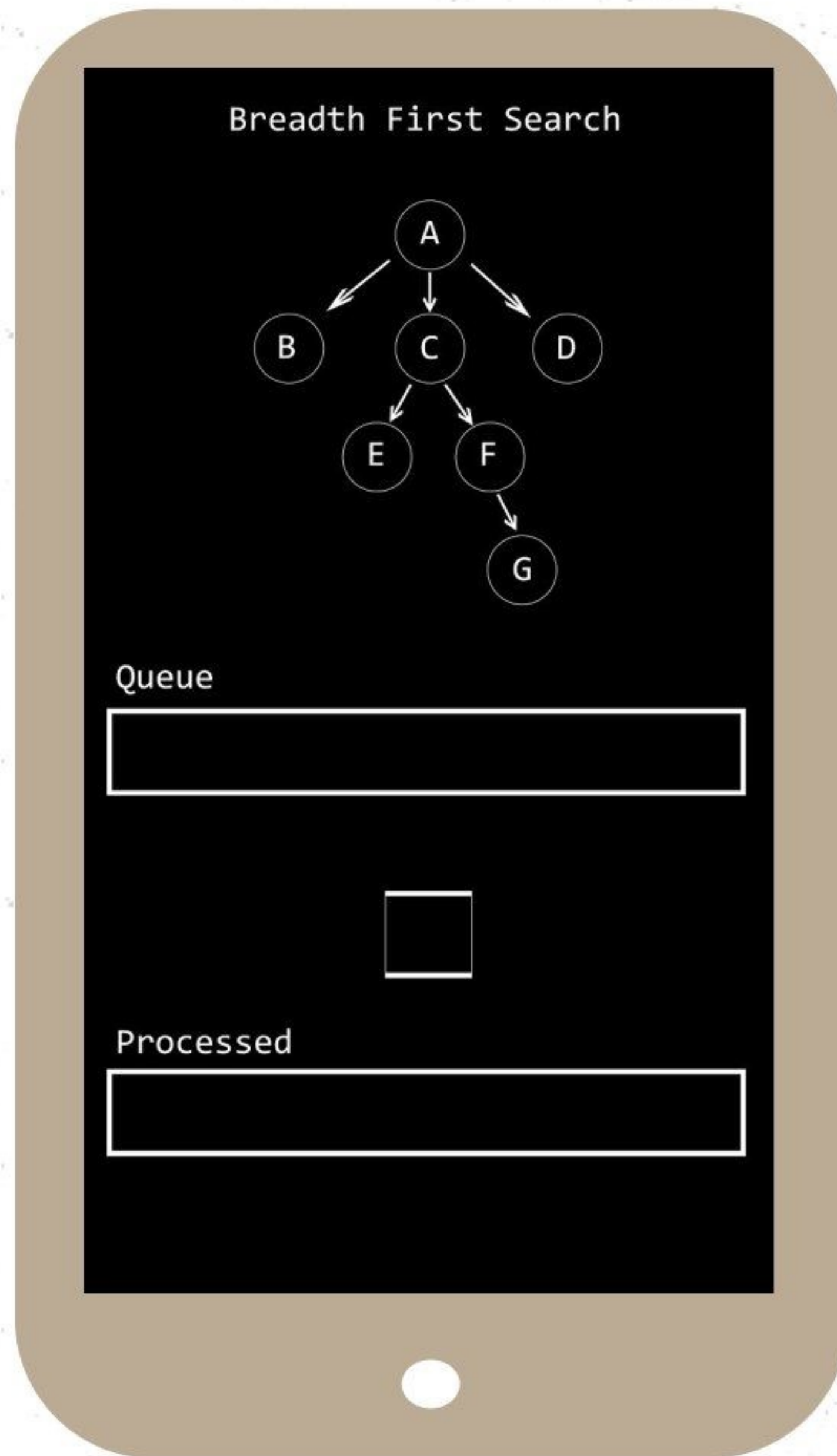
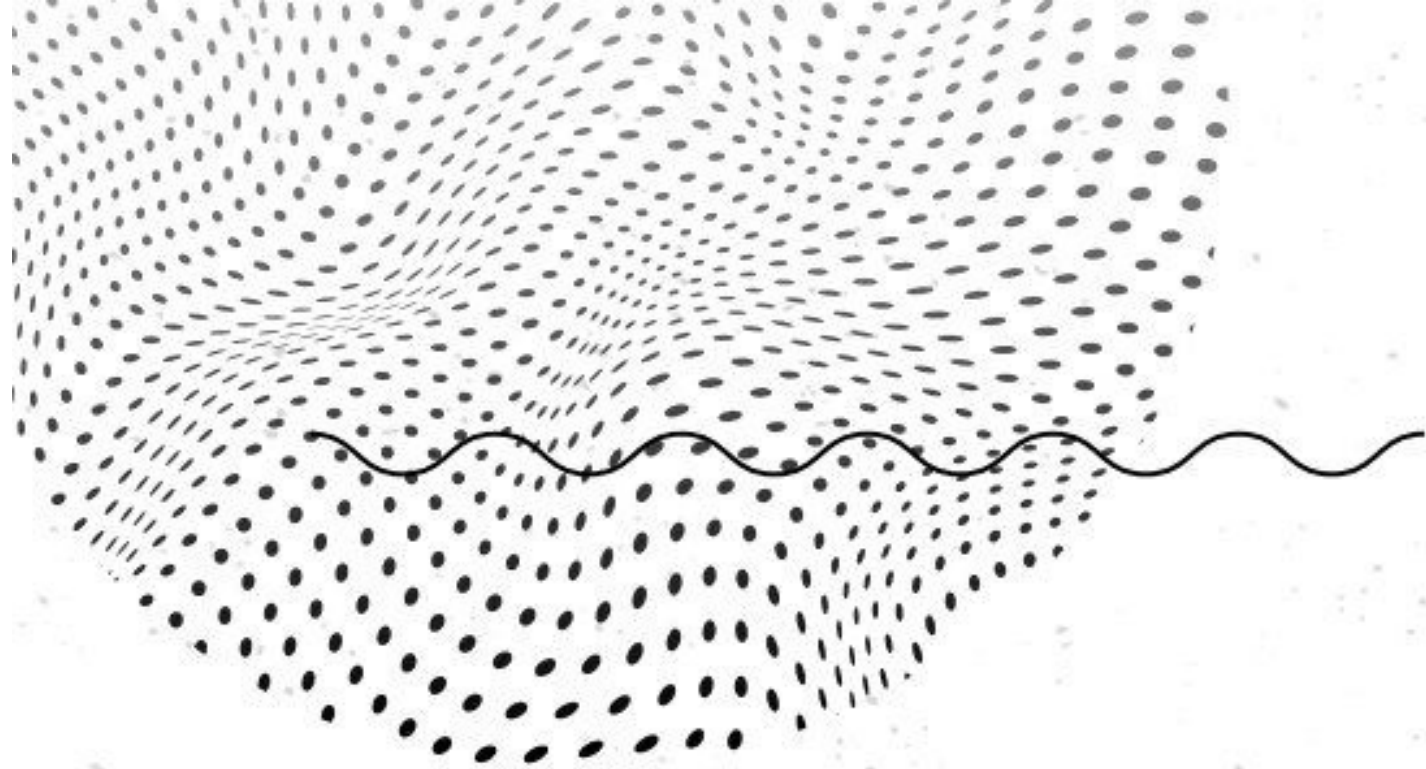
SEARCH

Graph traversal method
that explores all
neighbors level by level



< T 3 >







BF SEARCH Pseudocode

time complexity



BFS(graph, start):

- create an empty queue Q

- mark start as visited

- enqueue start into Q

- while Q is not empty:

 - current = dequeue Q

 - process(current) // e.g., print or store current node

 - for each neighbor of current:

 - if neighbor is not visited:

 - mark neighbor as visited

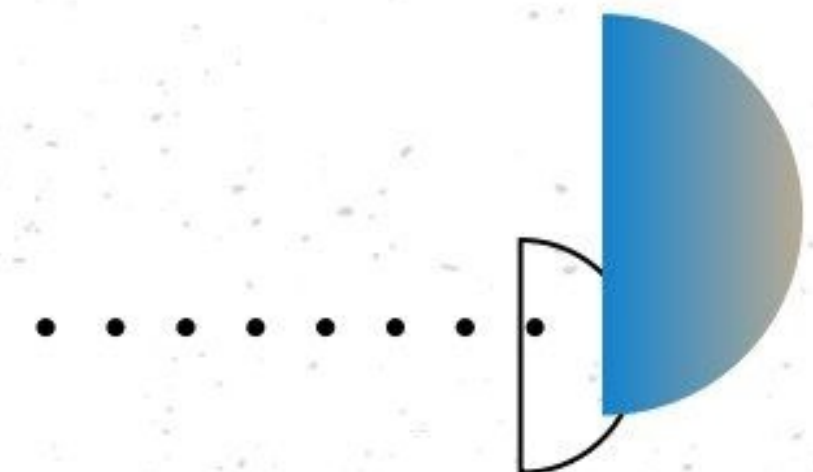
 - enqueue neighbor into Q

V = number of vertices (nodes)

E = number of edges (connections)

Time Complexity: $O(V + E)$ — each node and edge is visited once.

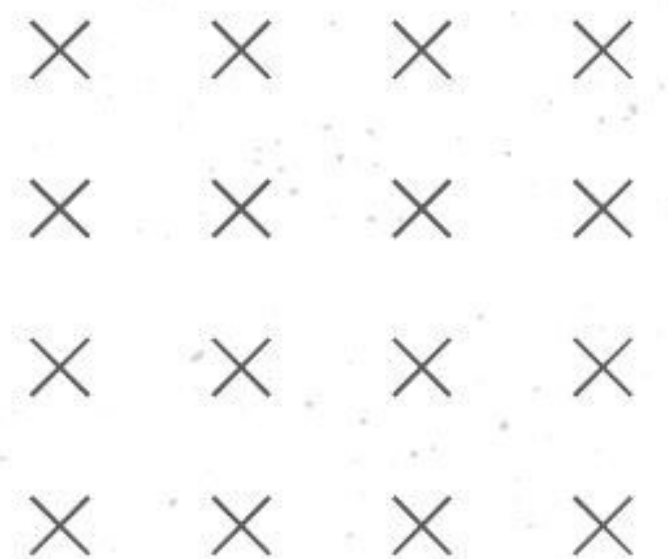
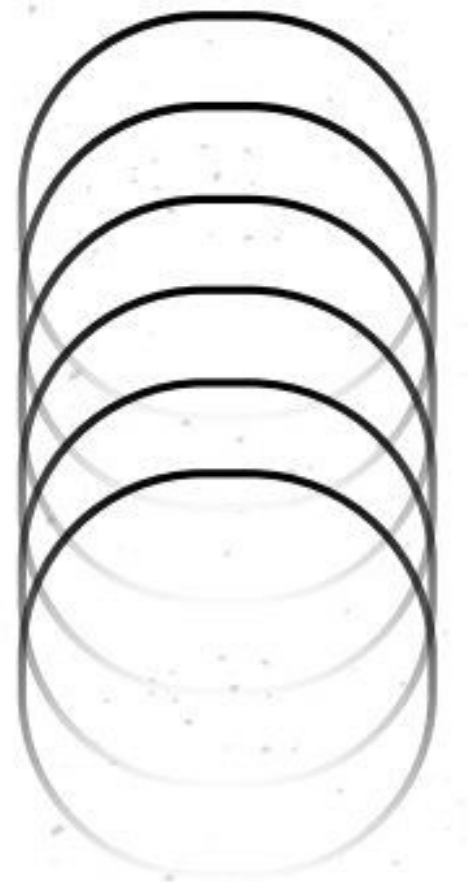
Space Complexity: $O(V)$ — for the queue and visited list.

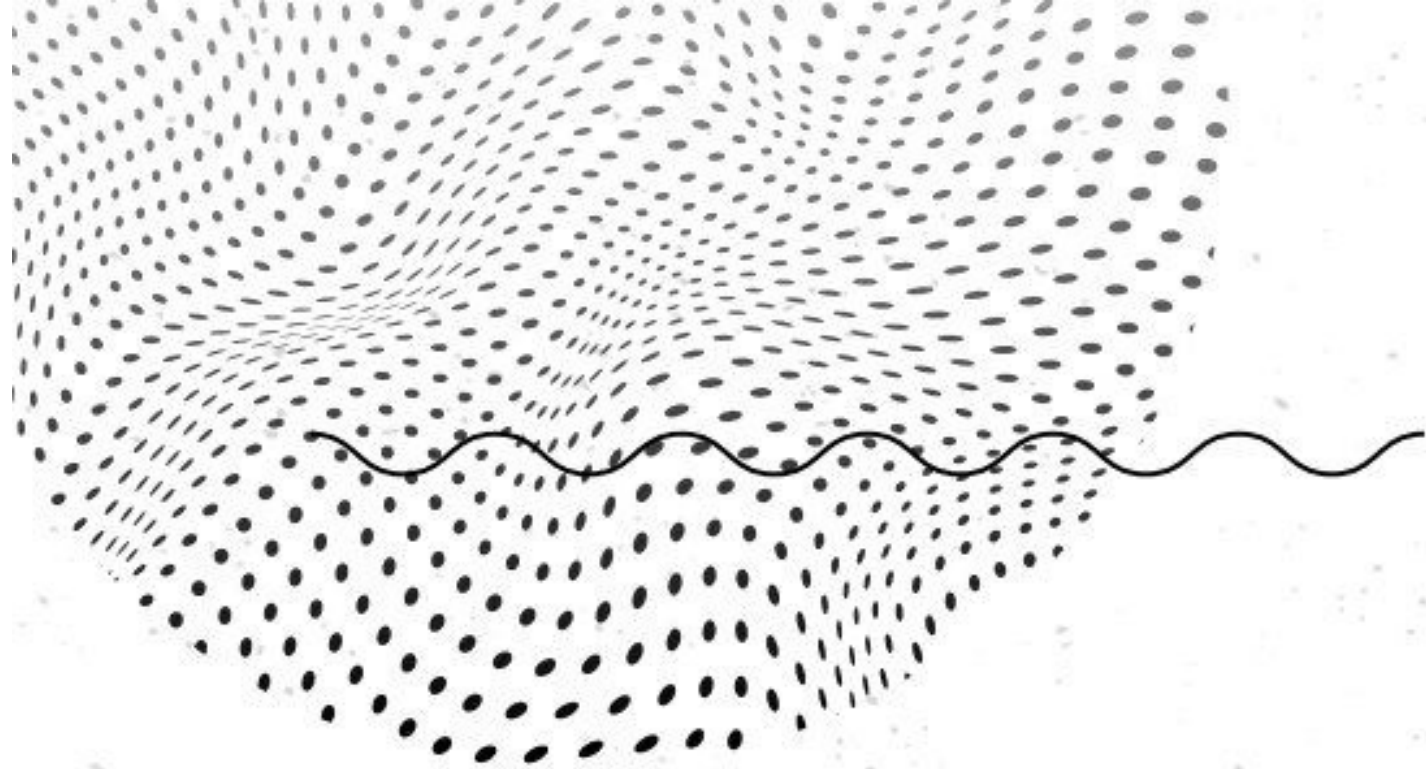


4. DF SEARCH

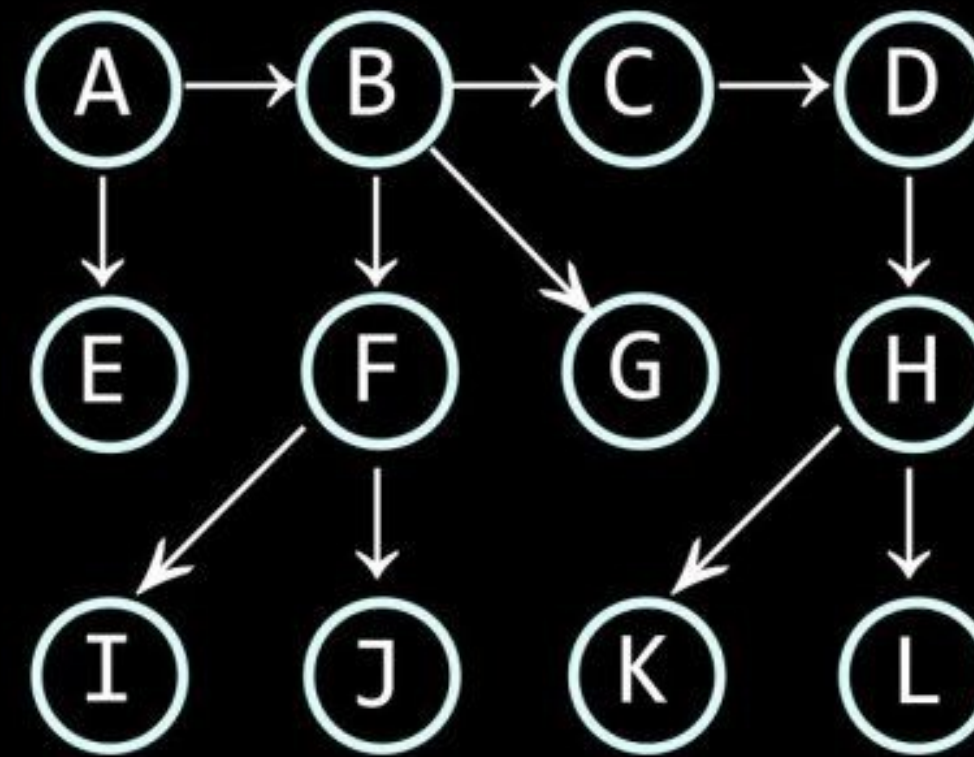
Graph traversal that
goes deep into each
branch before
backtracking

< T 4 >





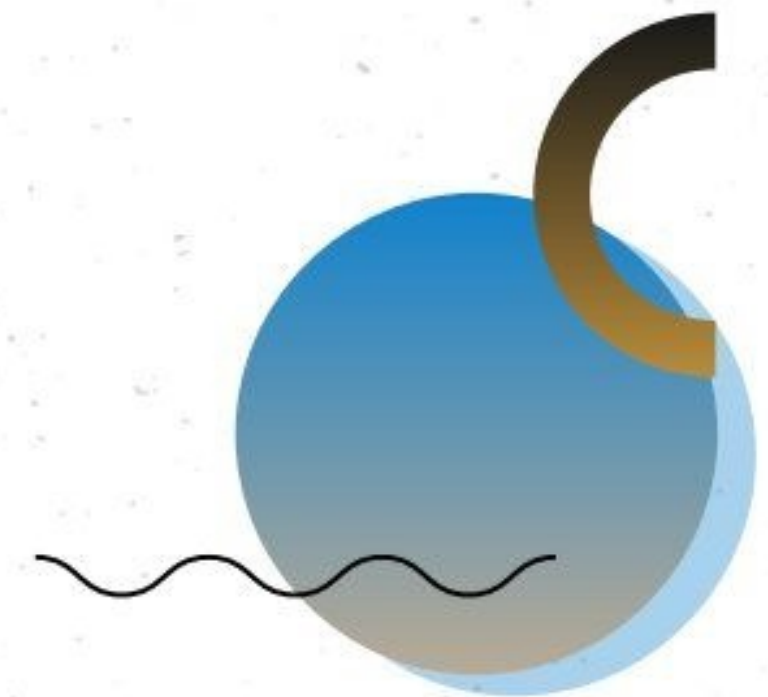
DEPTH FIRST SEARCH



Stack

Processed

Processing





DF SEARCH Pseudocode

time complexity

DFS(graph, start):

- create an empty stack S

- mark start as visited

- push start into S

- while S is not empty:

 - current = pop S

 - process(current) // e.g., print or store current node

 - for each neighbor of current in reverse order:

 - if neighbor is not visited:

 - mark neighbor as visited

 - push neighbor into S

Time Complexity of DFS

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$ (due to the stack and visited list)

